

Binary Trees

**Some updates are based on the presentation by Mr. Abdul Bari
on YouTube**

Tree Terminologies

- Root
- Parent
- Child
- Siblings
- Descendants
- Ancestors
- Degree of a node
- Internal/External
(Non-leaf/Leaf)
(Nonterminal/Terminal)
nodes
- Levels
- Height of a tree
- Forest

Height

0

1

2

3

4

Level

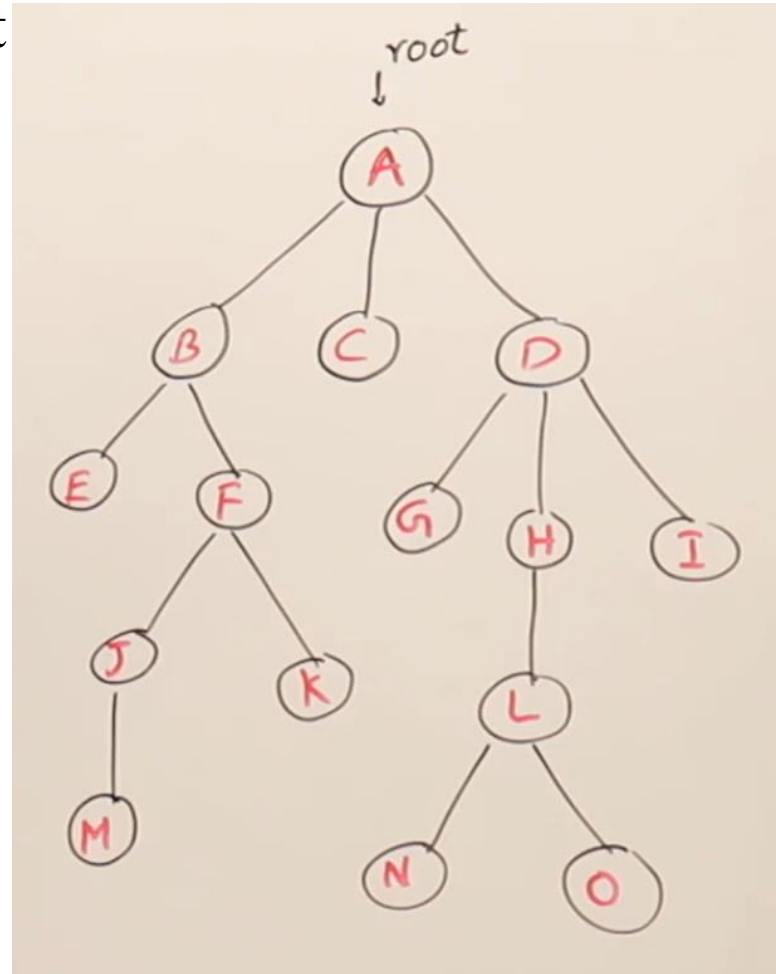
1

2

3

4

5



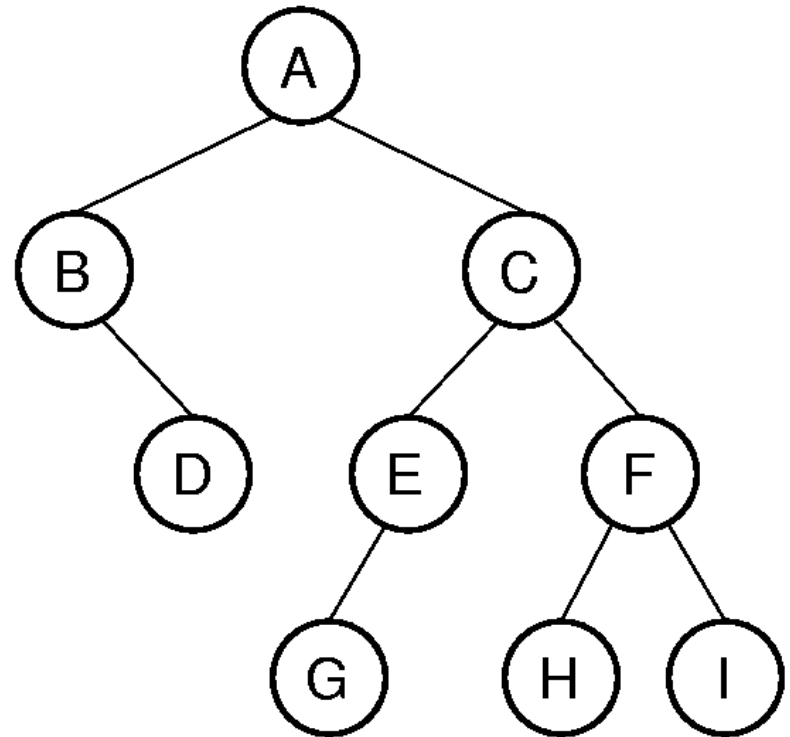
Binary Trees

A binary tree is made up of a finite set of nodes that is either empty or consists of a node called the root together with two binary trees, called the left and right subtrees, which are disjoint from each other and from the root.

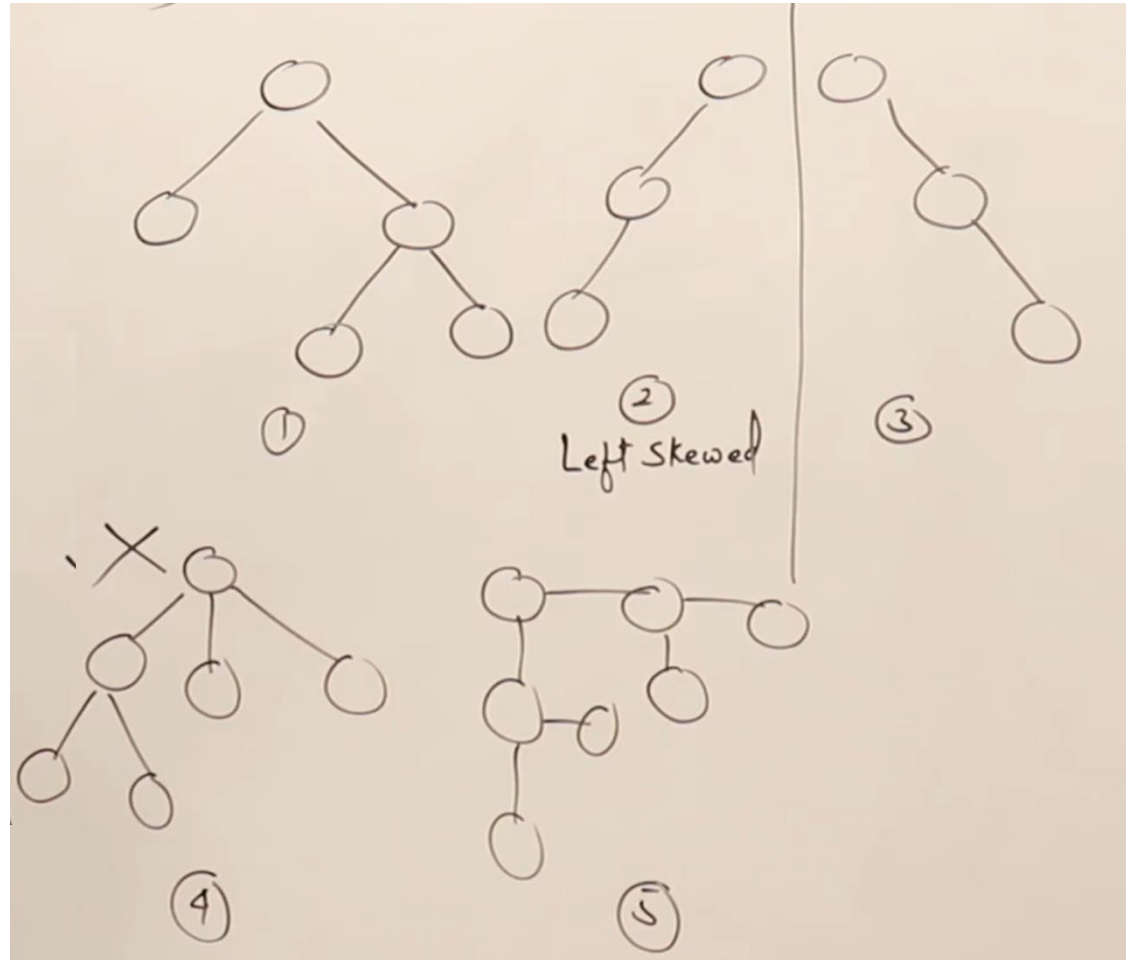
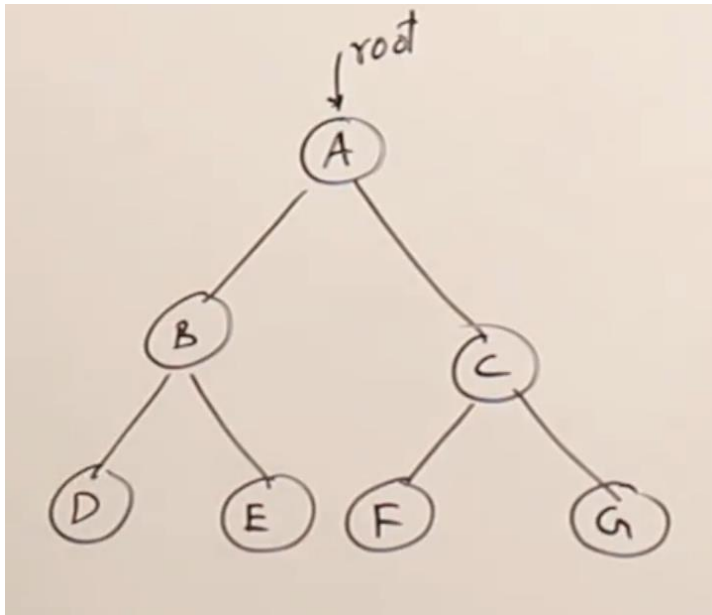
A simpler definition: A tree with degree of 2. Every node can have **maximum** 2 children.

Binary Tree Example

- $\deg(T) = 2$
- Every node can have maximum 2 children.
- The number of children can be 0, 1, or 2.
- Left child, right child



Binary Tree Example

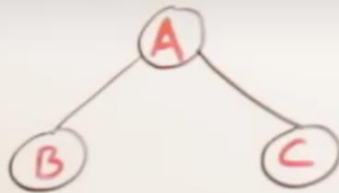


Traversals

Any process for visiting the nodes in some order is called a traversal.

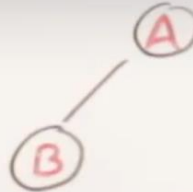
Any traversal that lists every node in the tree exactly once is called an enumeration of the tree's nodes.

Traversals



Pre: A, B, C
In: B, A, C
post: B, C, A
Level: A, B, C

Preorder: visit(node), Preorder(left subtree), Preorder(right subtree)



pre: A, B,
In: B, A,
post: B, A,
Level: A, B



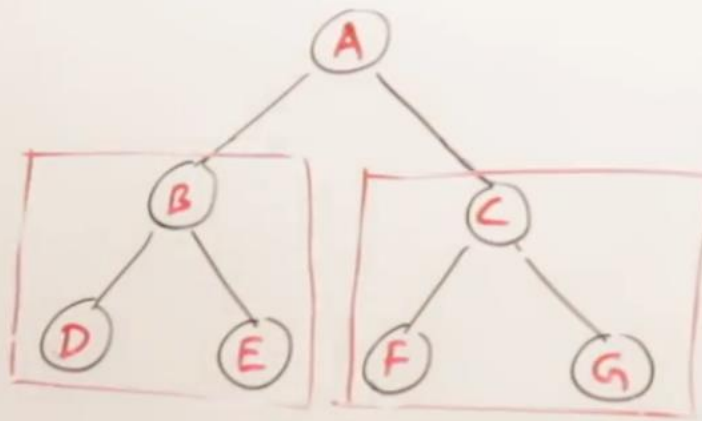
pre: A, B
In: A, B
post: B, A
Level: A, B

Inorder: Inorder(left), visit(node), Inorder(right)

Postorder: Postorder(left), Postorder(right), visit(node)

Level order: Level by Level

Traversals



Level: A, B, C, D, E, F, G

Pre: A, (B, D, E), (C, F, G)

A, B, D, E, C, F, G

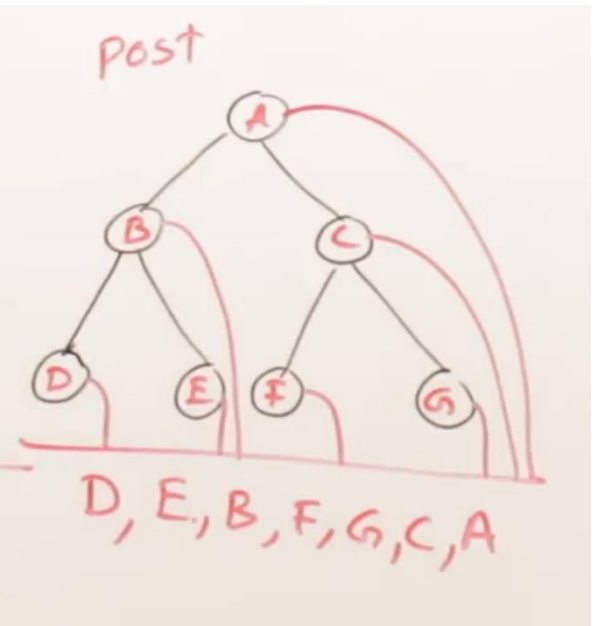
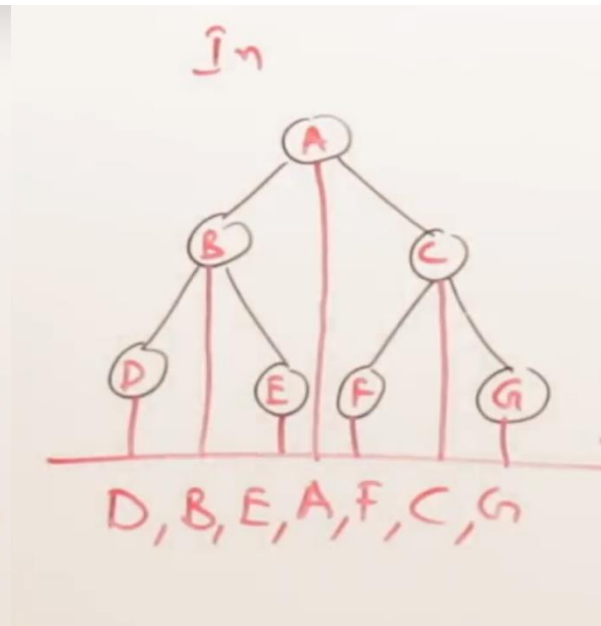
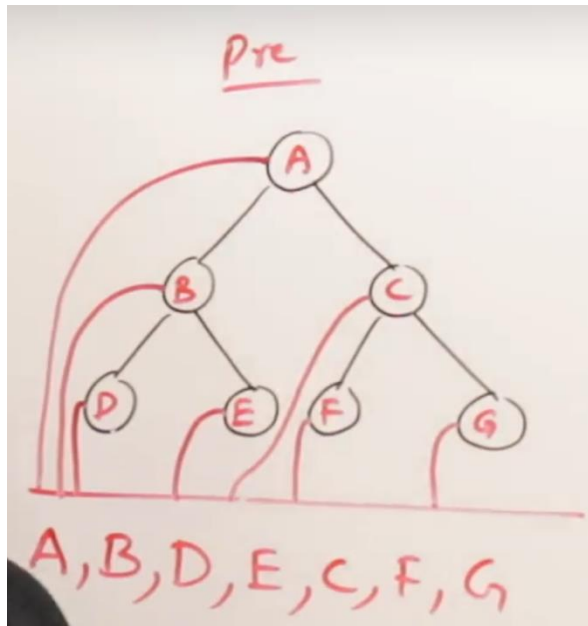
In: (D, B, E), A, (F, C, G)

D, B, E, A, F, C, G

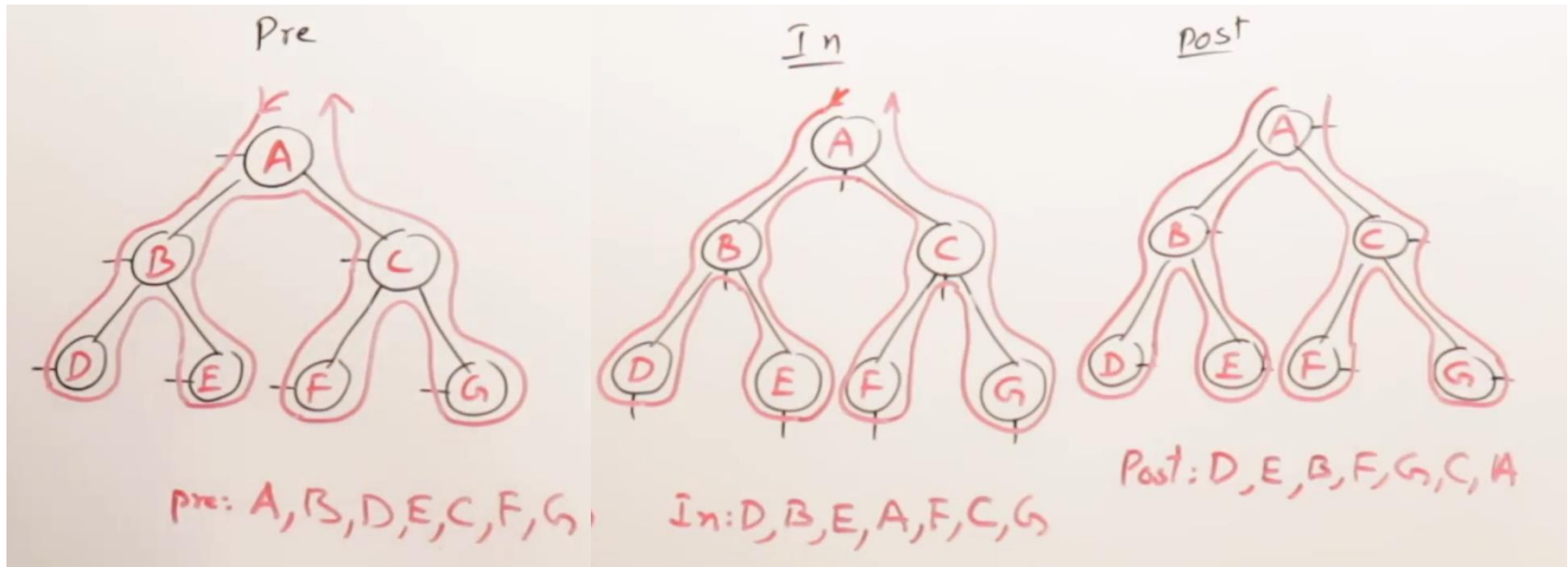
Post: (D, E, B), (F, G, C), A

D, E, B, F, G, C, A

Traversals (Easy Method #1)



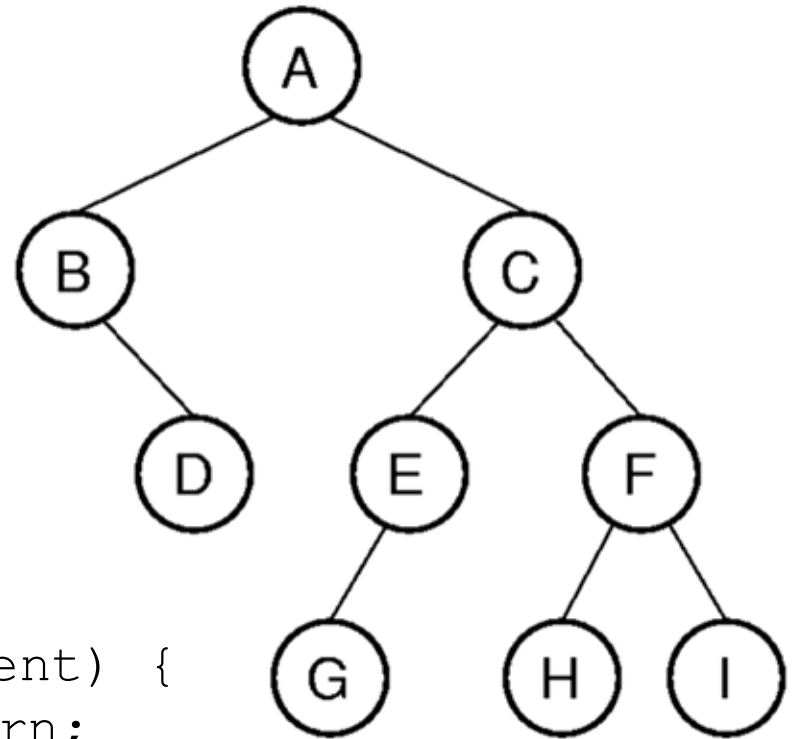
Traversals (Easy Method #2)



Preorder

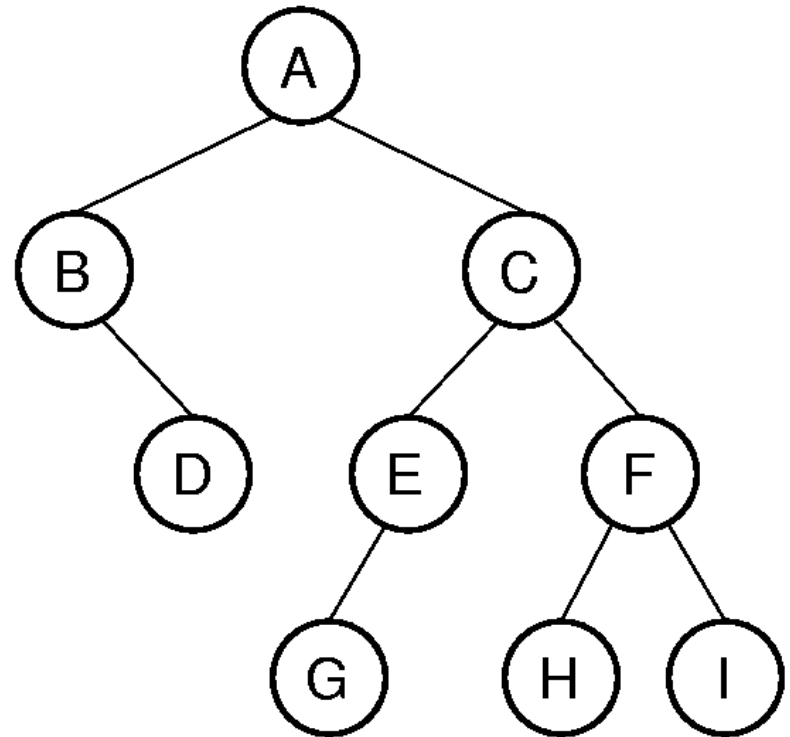
- Visit current node, then visit each child, left-to-right:
 - ABDCEGFHI
- Naturally recursive:

```
// preorder traversal
void preorder(BinNode* current) {
    if (current == NULL) return;
    visit(current);
    preorder(current->left());
    preorder(current->right());
}
```



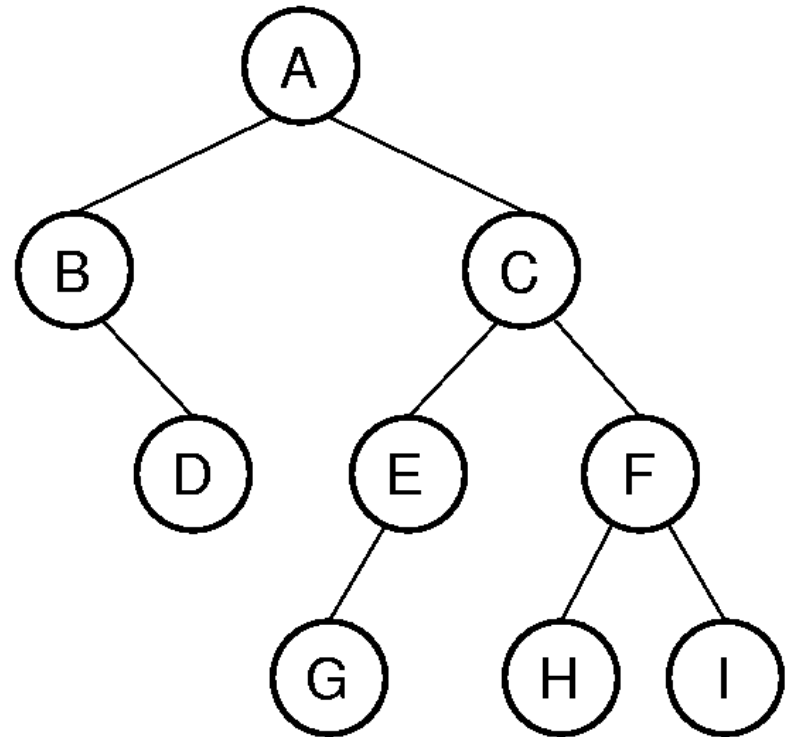
Postorder

- Visit each child, left-to-right, before the current node:
 - DBGEHIFCA



Inorder

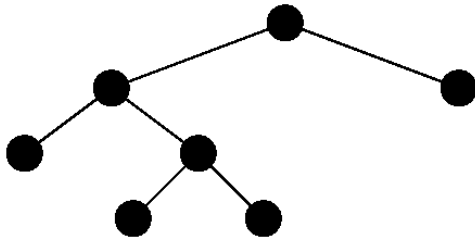
- Only makes sense for *binary* trees.
- This traversal assumes that each node has at most two children: a left and a right.
- Visit nodes in order: left child, current, right child.
 - BDAGECHFI



Full and Complete Binary Trees

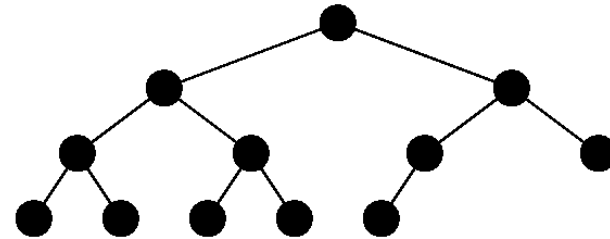
Full binary tree: Each node is either a leaf or internal node with exactly two non-empty children.

Complete binary tree: If the height of the tree is d , then all levels except possibly level d are completely full. The bottom level has all nodes to the left side.



(a)

Full, not complete



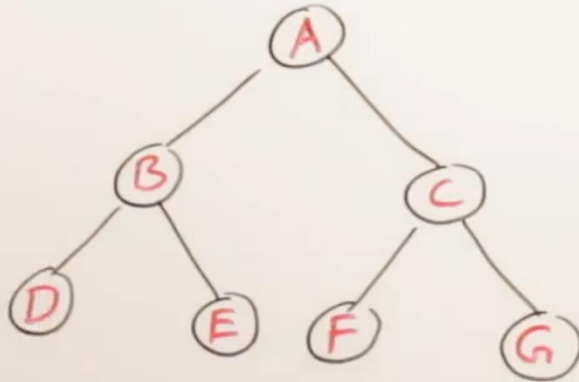
(b)

Complete, not full

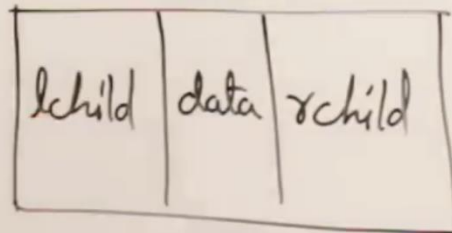
Age Group	Percentage
18-24	10%
25-34	20%
35-44	25%
45-54	20%
55-64	15%
65-74	10%
75-84	5%
85+	5%



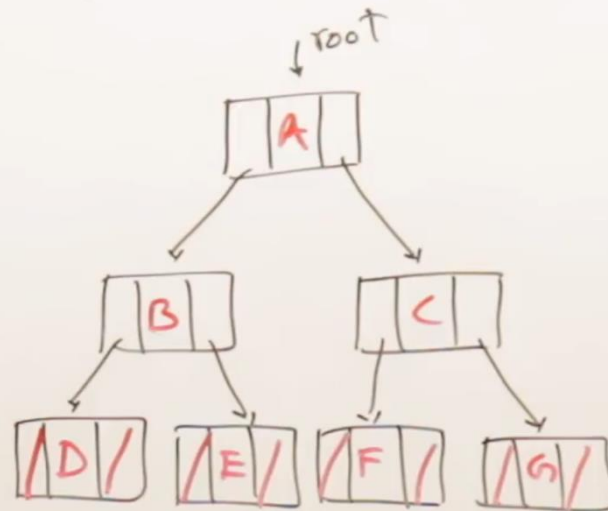
Linked Representation of Binary Trees



Node



```
struct Node  
{  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```



$$n = 7$$

$$n + 1 - \text{NULLs}$$

$$e = i + 1$$

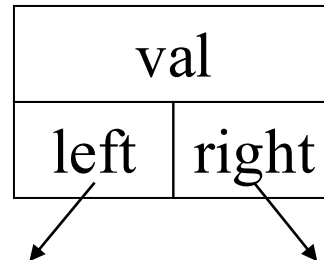
A Binary Tree Node ADT

Class BinNode

- Elem& val() – return the value of a node
- void setVal(const Elem&) – set the value
- BinNode* left() – return the left child
- BinNode* right() – return the right child
- void setLeft(BinNode*) – set the left child
- void setRight(BinNode*) – set the right child
- bool isLeaf() – return true if leaf node, else false

Representing Nodes

- Simplest node representation:



- All of the leaf nodes have two null pointers
- How much wasted space?
 - By our previous corollary, more empty pointers than nodes in the entire tree!
 - Sometimes leaf nodes hold more (all) data.

Representing Nodes (2)

- We can use inheritance to allow two kinds of nodes: internal and leaf nodes
- Base class: VarBinNode
 - isLeaf – pure virtual function
- Derived classes: IntlNode and LeafNode
- Typecast pointers once we know what kind of node we are working with...

Inheritance (1)

```
class VarBinNode {    // Abstract base class
public:
    virtual bool isLeaf() = 0;
};
```

```
class LeafNode : public VarBinNode { // Leaf
private:
    Operand var;                // Operand value
public:
    LeafNode(const Operand& val)
        { var = val; } // Constructor
    bool isLeaf() { return true; }
    Operand value() { return var; }
};
```

Inheritance (2)

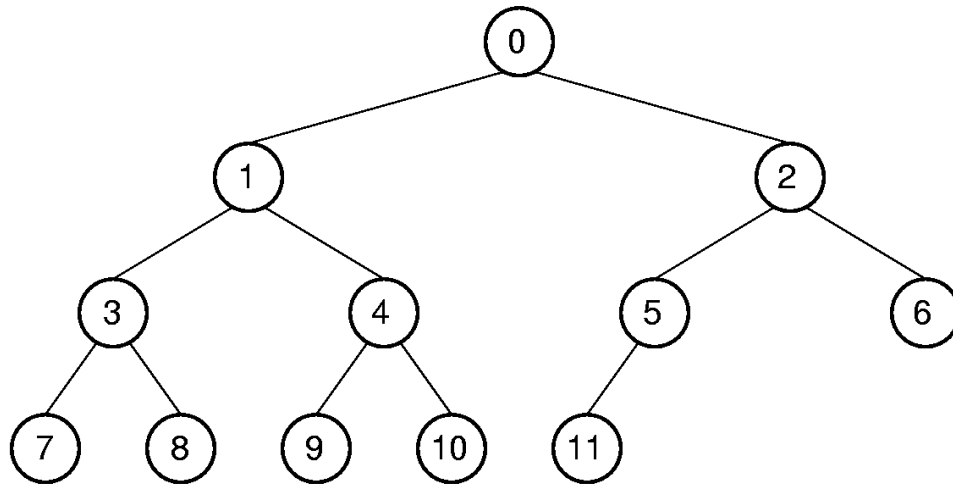
```
// Internal node
class IntlNode : public VarBinNode {
private:
    VarBinNode* left;           // Left child
    VarBinNode* right;          // Right child
    Operator opx;                // Operator value
public:
    IntlNode(const Operator& op,
              VarBinNode* l, VarBinNode* r)
        { opx = op; left = l; right = r; }
    bool isLeaf() { return false; }
    VarBinNode* leftchild() { return left; }
    VarBinNode* rightchild() { return right; }
    Operator value() { return opx; }
};
```

Inheritance (3)

```
// Preorder traversal
void traverse(VarBinNode *subroot) {
    if (subroot == NULL) return; // Empty
    if (subroot->isLeaf())        // Do leaf node
        cout << "Leaf: "
              << ((LeafNode *) subroot)->value()
              << endl;
    else {                        // Do internal node
        cout << "Internal: "
              << ((IntlNode *) subroot)->value()
              << endl;
        traverse(((IntlNode *) subroot)->
                leftchild());
        traverse(((IntlNode *) subroot)->
                rightchild());
    }
}
```

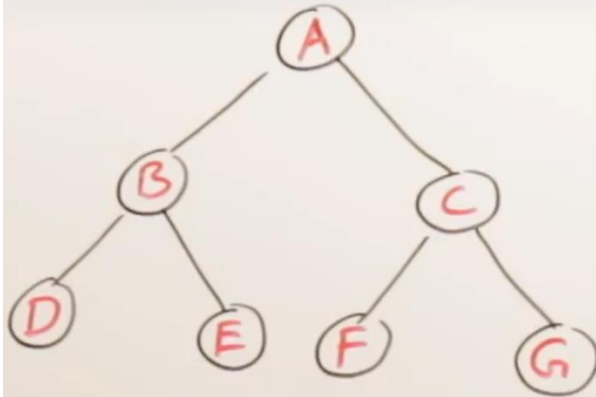
Complete Binary Trees

- For a complete tree, we can save a lot of space by storing the tree in an array (no pointers!)

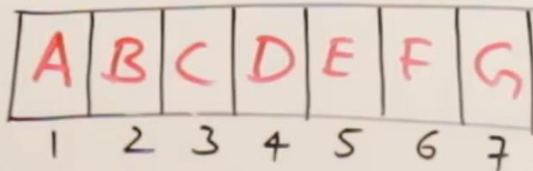


0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

Array Representation of Binary Trees



element - i
Lchild - $2*i$
Rchild - $2*i+1$
Parent - $\left\lfloor \frac{i}{2} \right\rfloor$

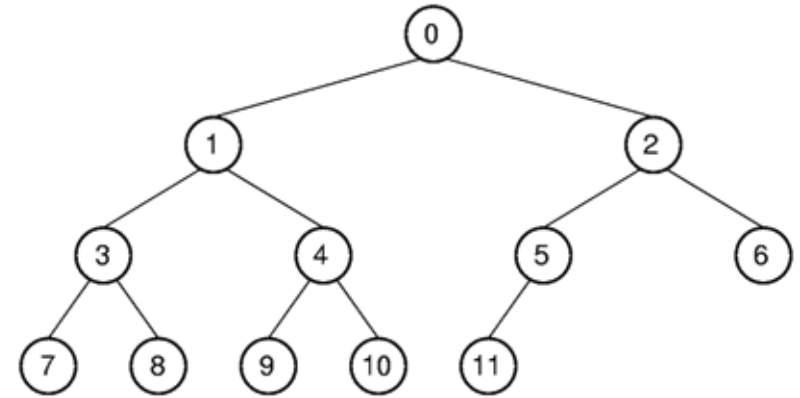


element	index	Lchild	Rchild
A	1	2	3
B	2	4	5
C	3	6	7
	i	$2*i$	$2*i+1$

- ❑ When the index starts from 1.
- ❑ We insert the elements into the array level-by-level.

Array Representation

How can we find parents, children, siblings?



Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

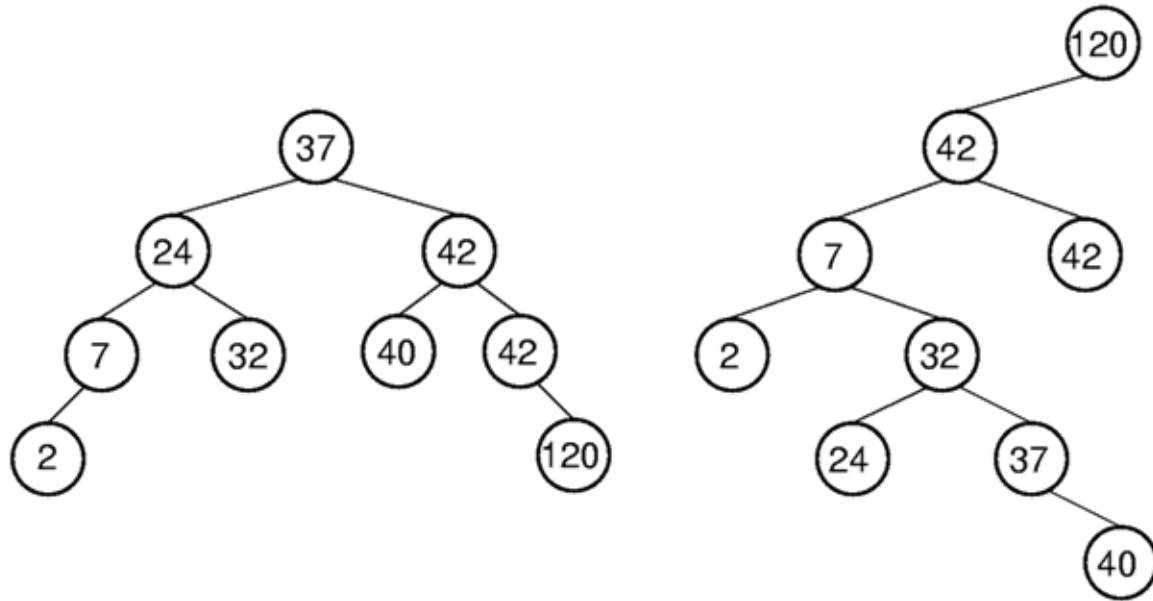
Array Relationships

- $\text{Parent}(r) = \lfloor (r-1)/2 \rfloor$ if $r \neq 0$
- $\text{LeftChild}(r) = 2r + 1$ if $2r + 1 < n$
- $\text{RightChild}(r) = 2r + 2$ if $2r + 2 < n$
- $\text{LeftSibling}(r) = r - 1$ if r is even
- $\text{RightChild}(r) = r + 1$ if r is odd and $r + 1 < n$

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

Binary Search Trees

- BST Property: All elements stored in the left subtree of a node with value K have values $< K$. All elements stored in the right subtree of a node with value K have values $\geq K$.



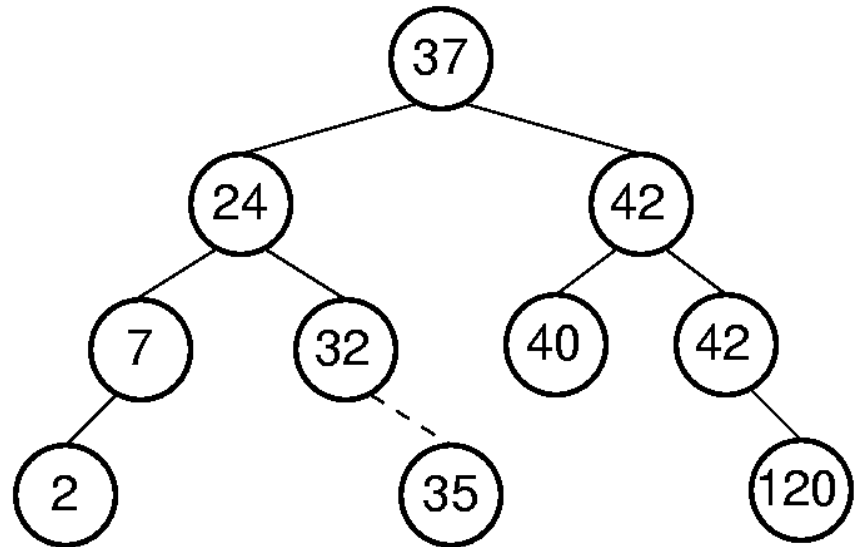
Searching the tree

```
Bool find(const Key& K, Elem& e) const  
{ return findhelp(root, K, e); }
```

```
template <class Key, class Elem,  
          class KEComp, class EEComp>  
bool BST<Key, Elem, KEComp, EEComp>::  
    findhelp(BinNode<Elem>* subroot,  
              const Key& K, Elem& e) const {  
    if (subroot == NULL) return false;  
    else if (KEComp::lt(K, subroot->val()))  
        return findhelp(subroot->left(), K, e);  
    else if (KEComp::gt(K, subroot->val()))  
        return findhelp(subroot->right(), K, e);  
    else { e = subroot->val(); return true; }  
}
```

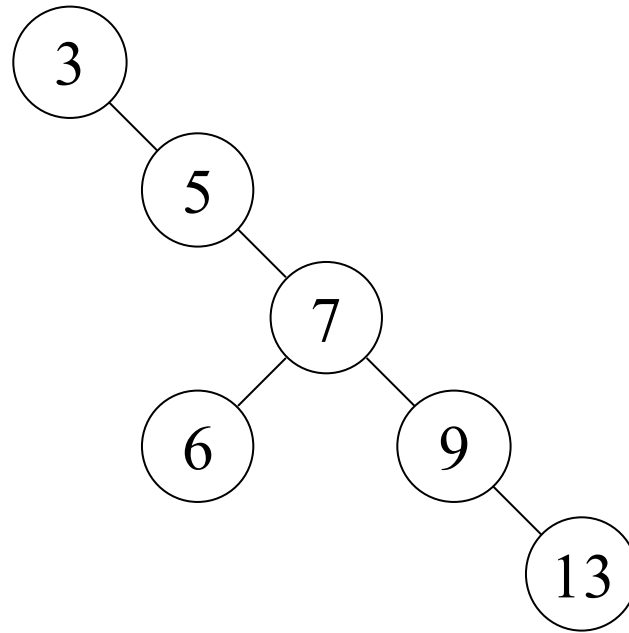
Inserting into the tree

- Find an appropriate leaf node, or internal node with no left/right child
- Insert the new value
- Can cause the tree to become unbalanced



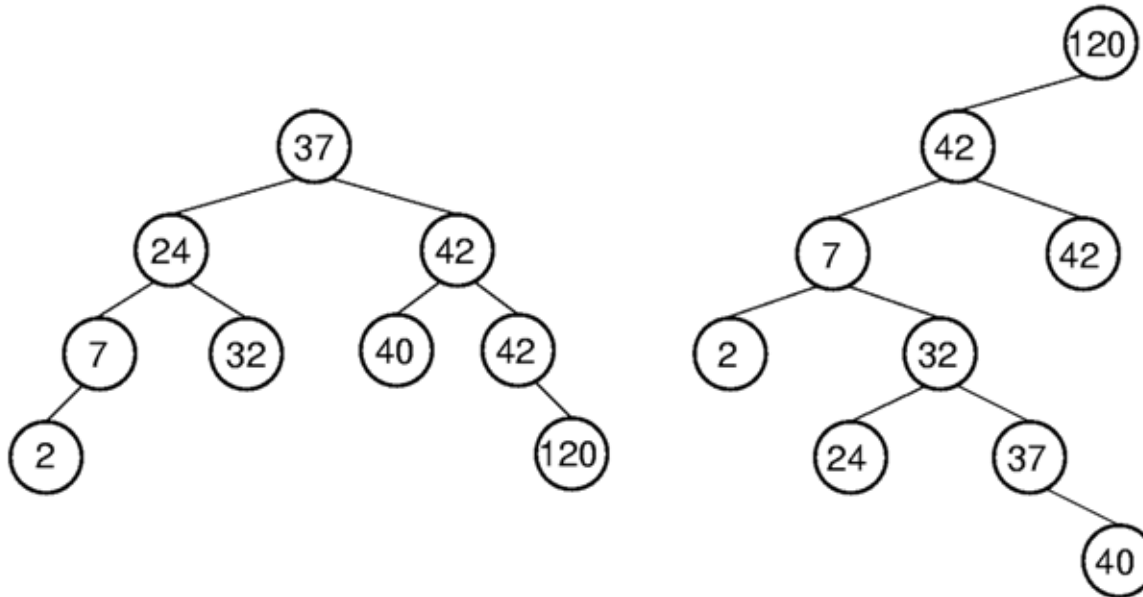
An unbalanced tree

- Suppose we insert 3, 5, 7, 9, 13, and 6



Cost of search

- Worst case cost = depth of tree
 - Worst case: tree linear, cost = n
 - Best case: perfect balance, cost = $\lg(n)$



BST insert

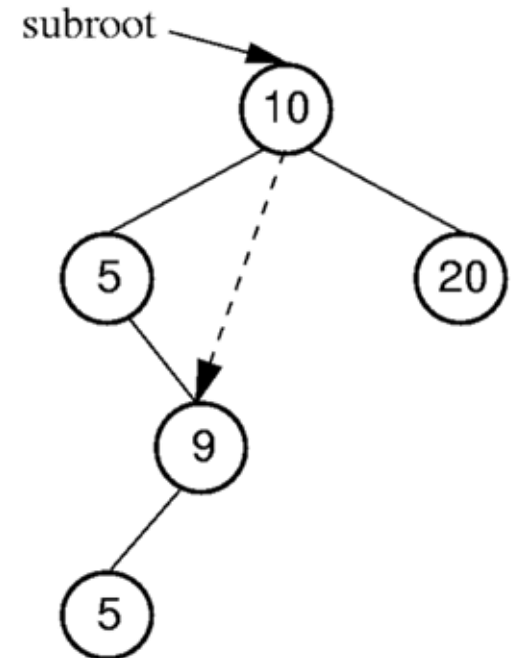
```
template <class Key, class Elem,
          class KECmp, class EECmp>
BinNode<Elem>* BST<Key,Elem,KECmp,EECmp>::
inserthelp(BinNode<Elem>* subroot,
            const Elem& val) {
    if (subroot == NULL) // Empty: create node
        return new BinNodePtr<Elem>(val,NULL,NULL);
    if (EECmp::lt(val, subroot->val()))
        subroot->setLeft(inserthelp(subroot->left(),
                                     val));
    else subroot->setRight(
        inserthelp(subroot->right(), val));
    // Return subtree with node inserted
    return subroot;
}
```


Deleting

- When deleting from a BST we must take care that...
 - The resulting tree is still a binary tree
 - The BST property still holds
- To start with, consider the case of deleting the *minimum* element of a subtree

Removing the minimal node

1. Move left until you can't any more
 - Call the minimal node S
2. Have the parent of S point to the right child of S
 - There was no left child, or we would have taken that link
 - Still less than S 's parent, so BST property maintained
 - NULL links are ok



Removing the min

```
...deletemin (BinNode<Elem>*  subroot,  
              BinNode<Elem>* & min) {  
    if (subroot->left() == NULL) {  
        min = subroot;  
        return subroot->right();  
    }  
    else { // Continue left  
        subroot->setLeft(  
            deletemin(subroot->left(), min));  
        return subroot;  
    }  
}
```

DeleteMin

--

10

8

5

5

8

10

```
deletemin(10)
```

```
    setleft(deletemin(8))
```

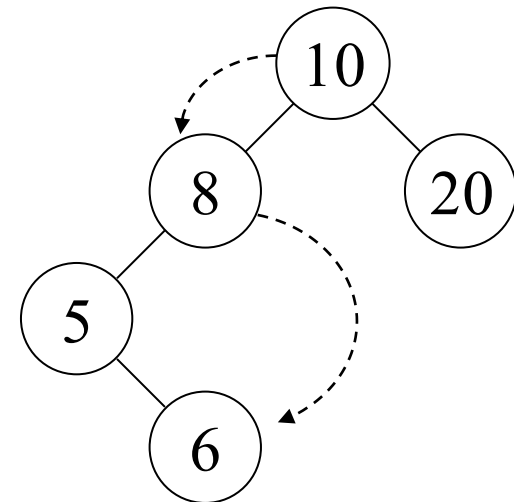
```
        setleft(deletemin(5))
```

```
            min = 5
```

```
            return 6
```

```
        return 8
```

```
    return 10
```

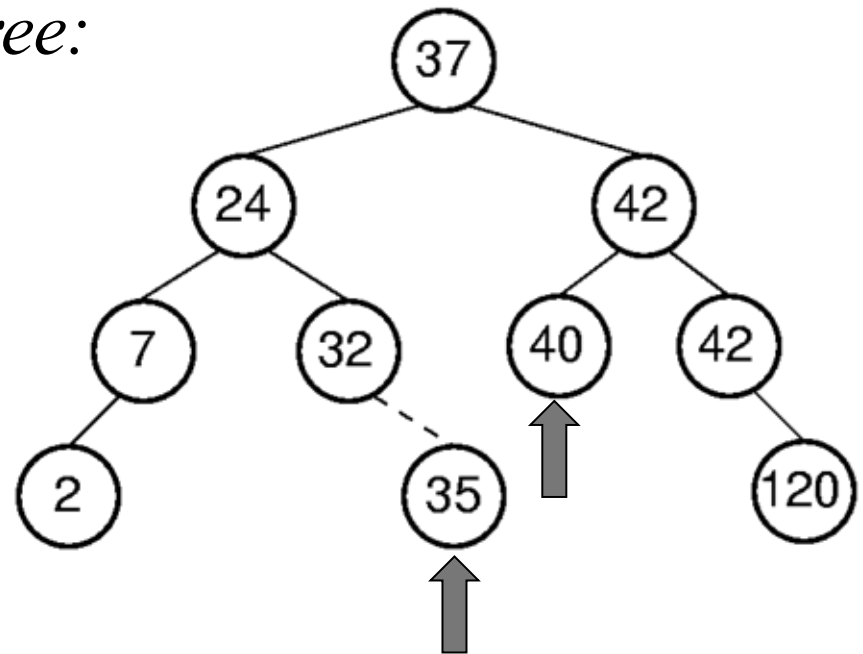


Deleting an arbitrary node

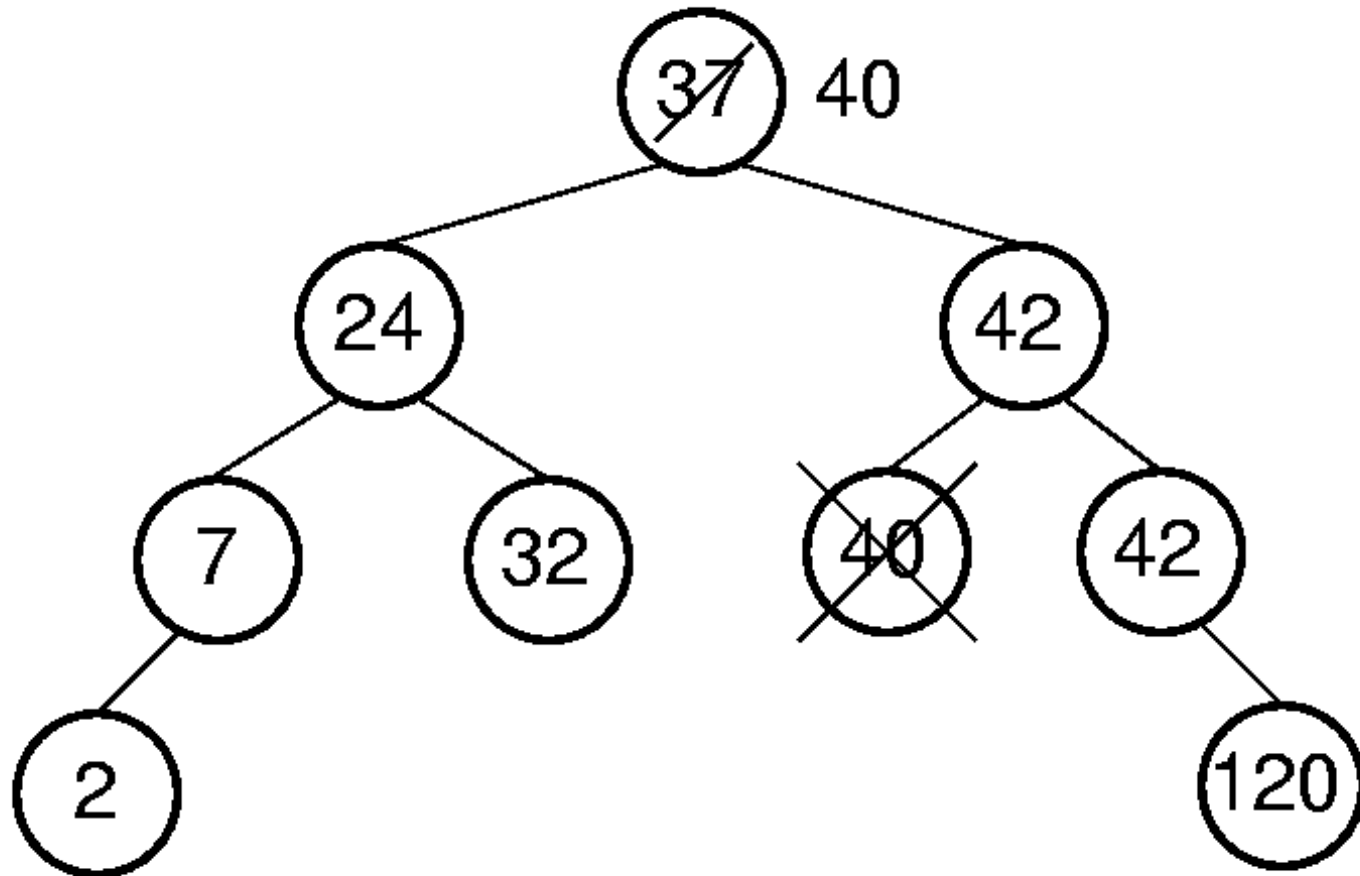
- If we delete an arbitrary node, R , from a BST, there are three possibilities:
 - R has no children – set it's parent's pointer to null
 - R has one child – set the parent's pointer to point to the child, similar to deletemin
 - R has two children – Now what?
 - *We have to find a node from R 's subtree to replace R , in order to keep a binary tree.*
 - *We must be careful to maintain the BST property*

Which node?

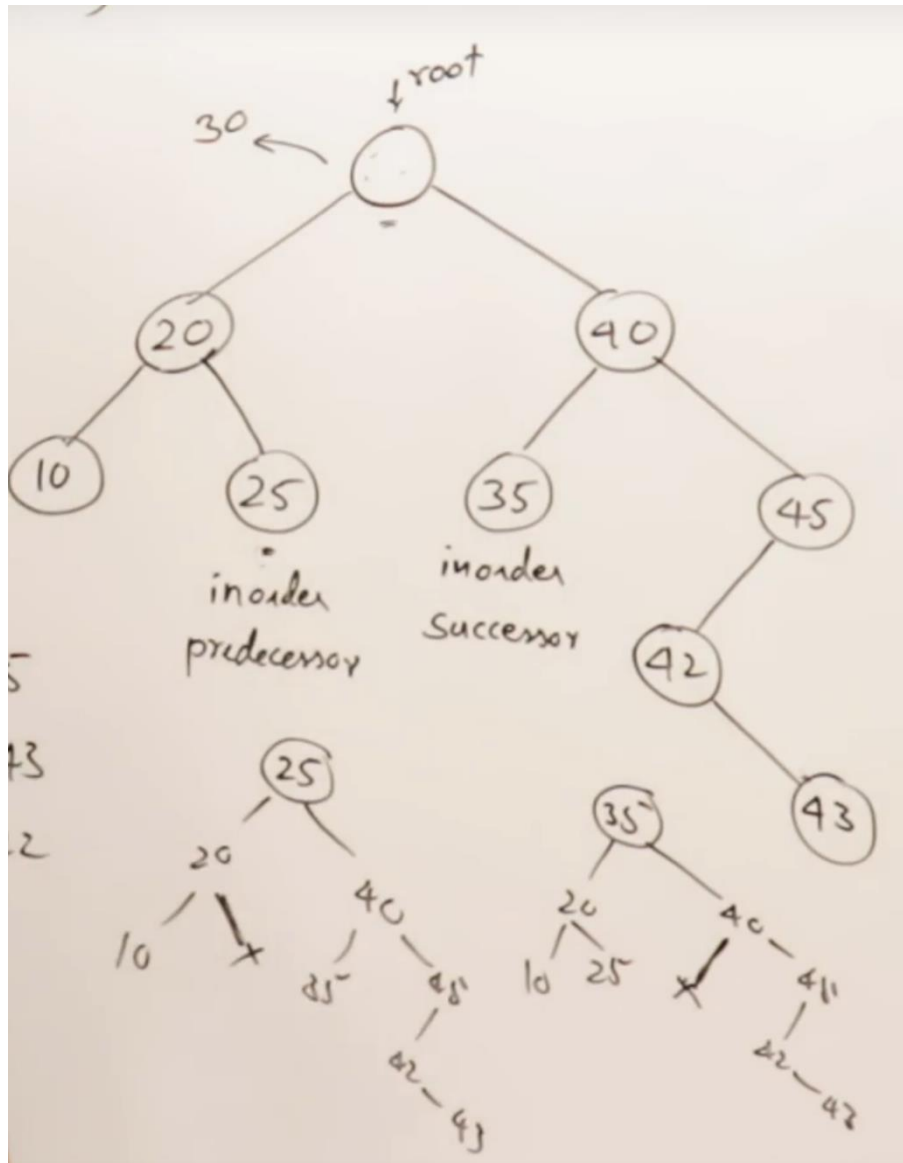
- Which node can we use to replace R ?
 - Which node in the tree will be most similar to R ?
 - *Depends on which subtree:*
 - Left subtree: rightmost
 - Everything in the right subtree is greater, everything in the left subtree is smaller
 - Right subtree: leftmost
 - Everything in the left subtree is smaller, everything in the right subtree is greater



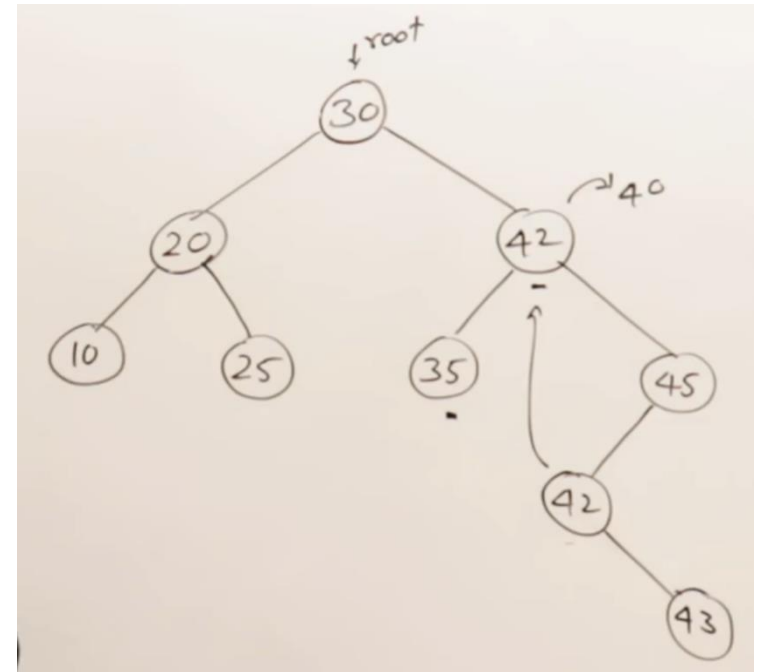
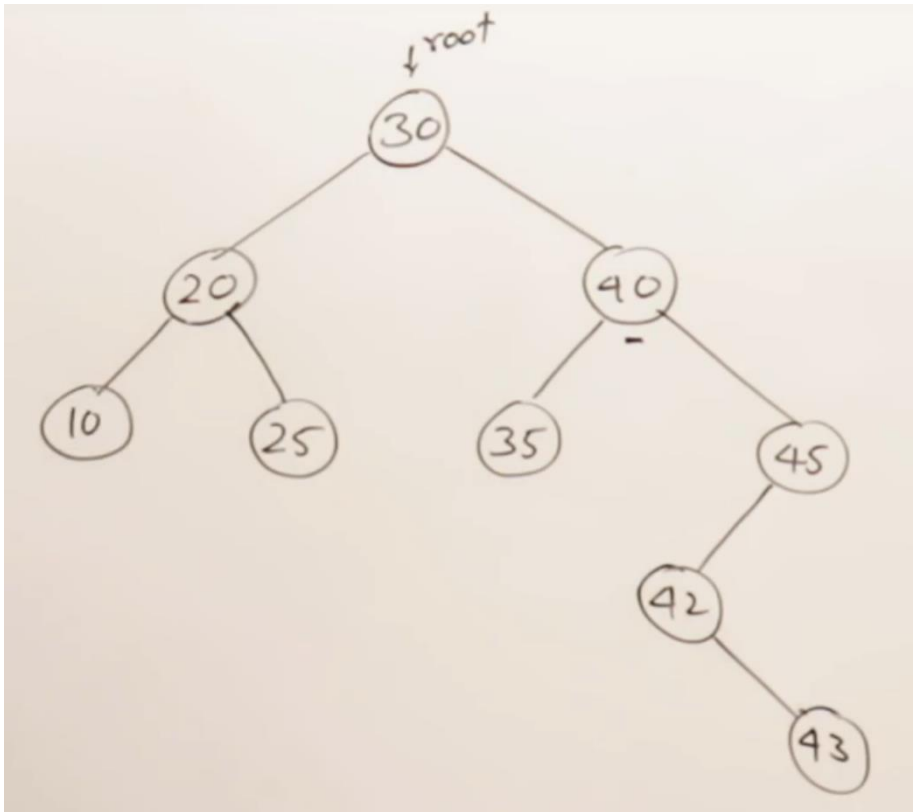
BST delete



BST delete



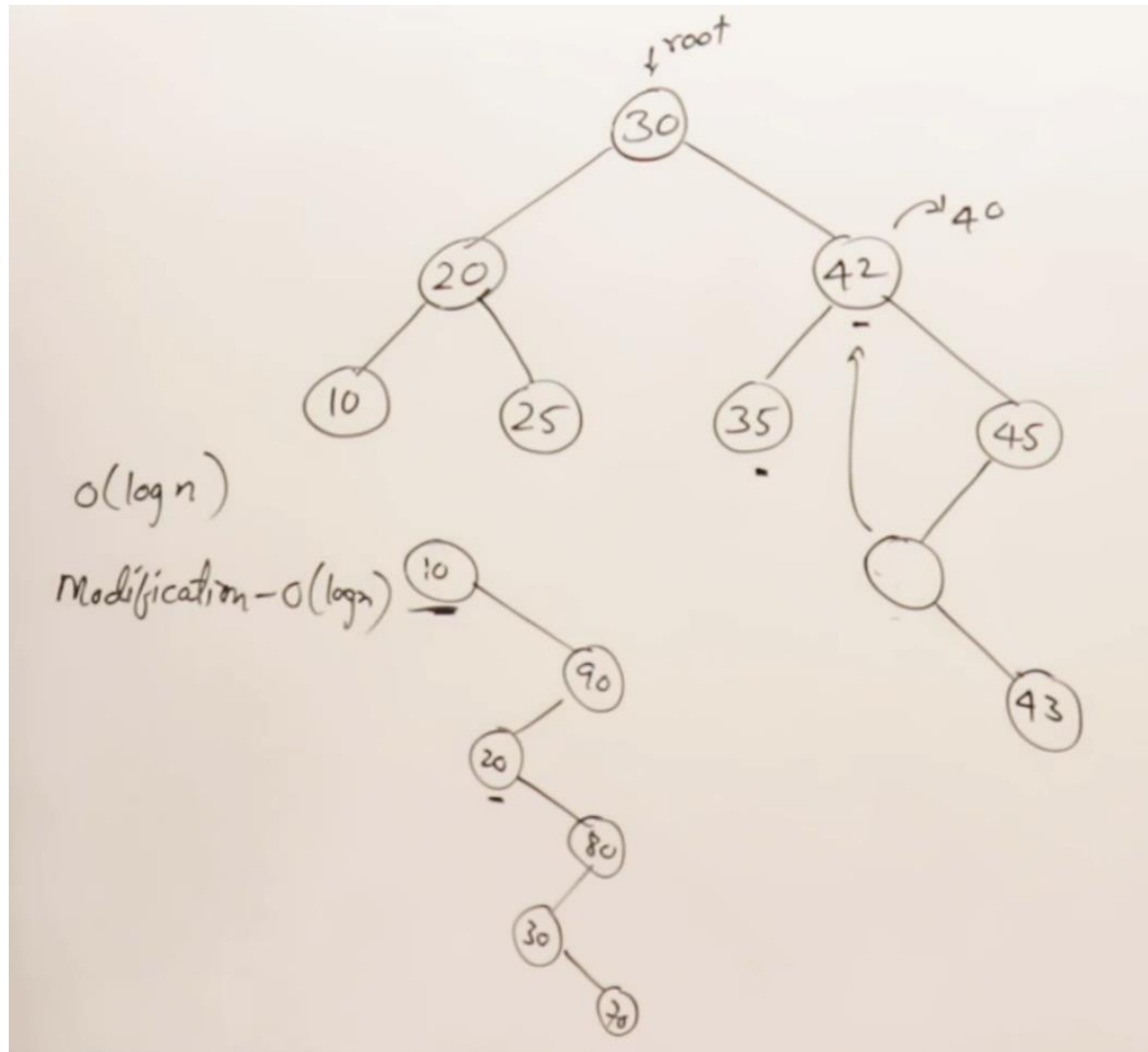
BST delete



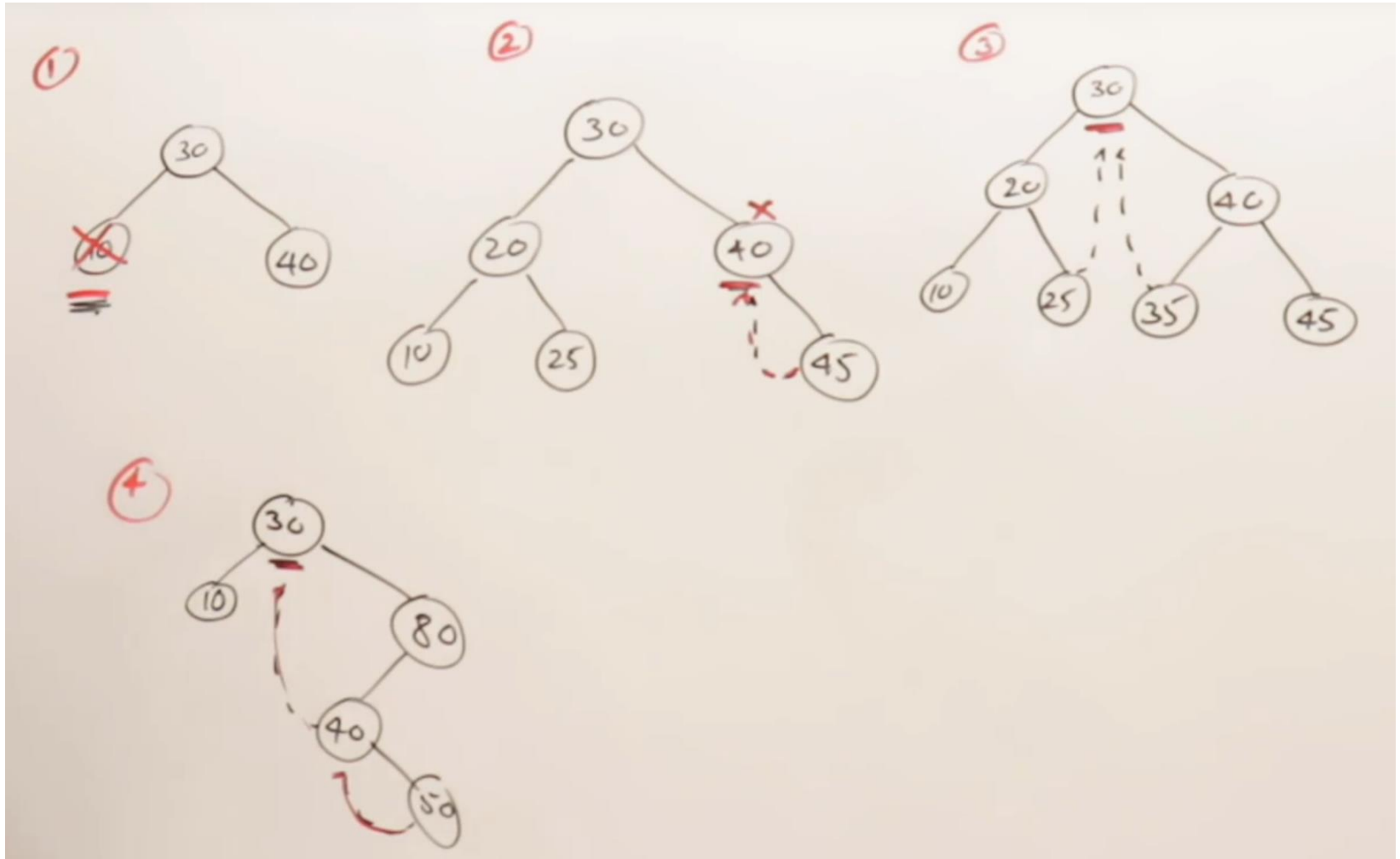
Which node to replace if we delete 40?

42 is not a leaf node; its place should be filled with its child 43.

BST delete



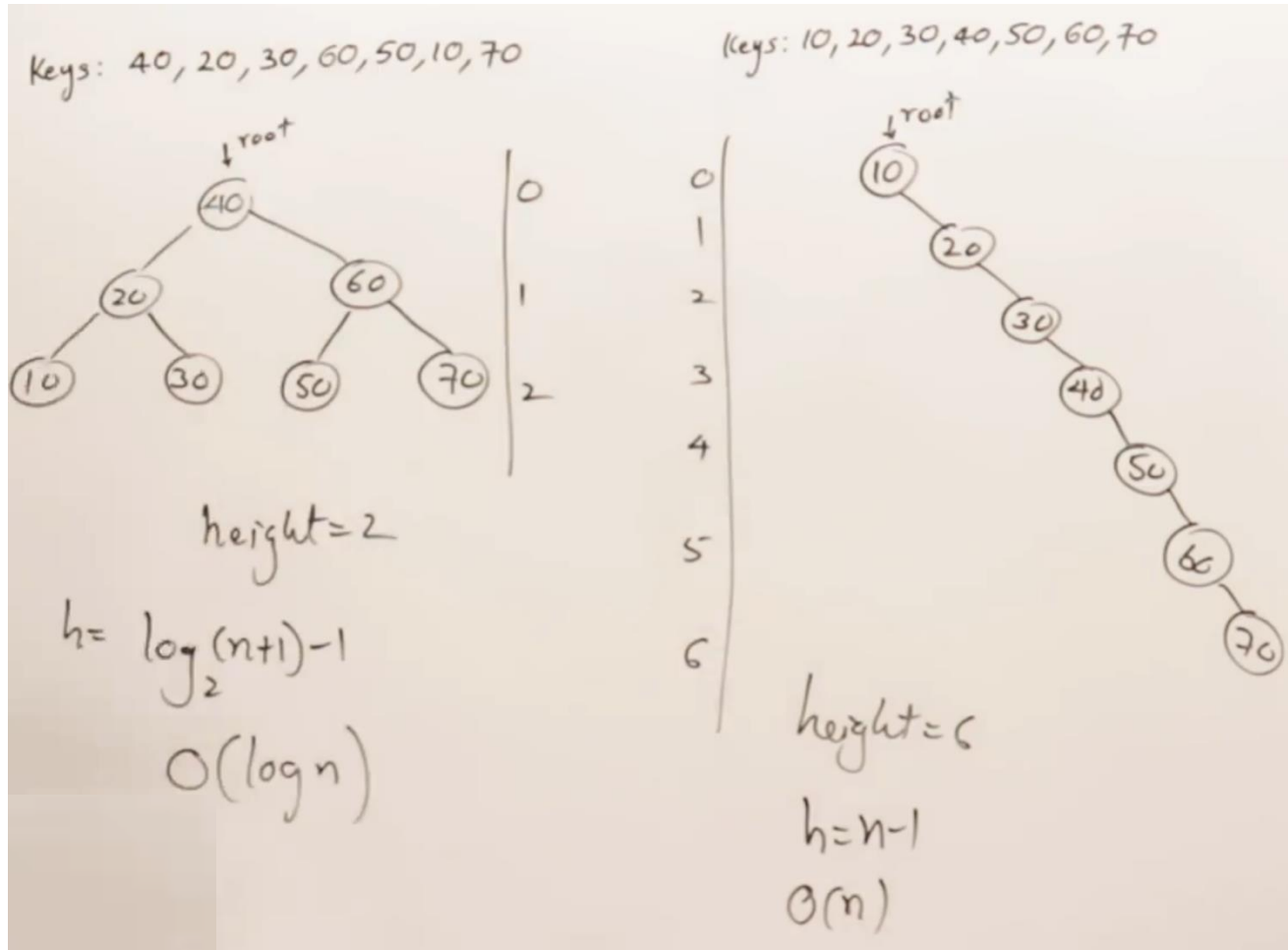
BST delete (Summary)



Duplicate values

- If there are no duplicates, either will work
- Duplicates
 - Recall that the left subtree has values $< K$, while the right has values $\geq K$.
 - Duplicates of K must be in the right subtree, so we must choose from the right subtree if duplicates are allowed.

Drawback of BST

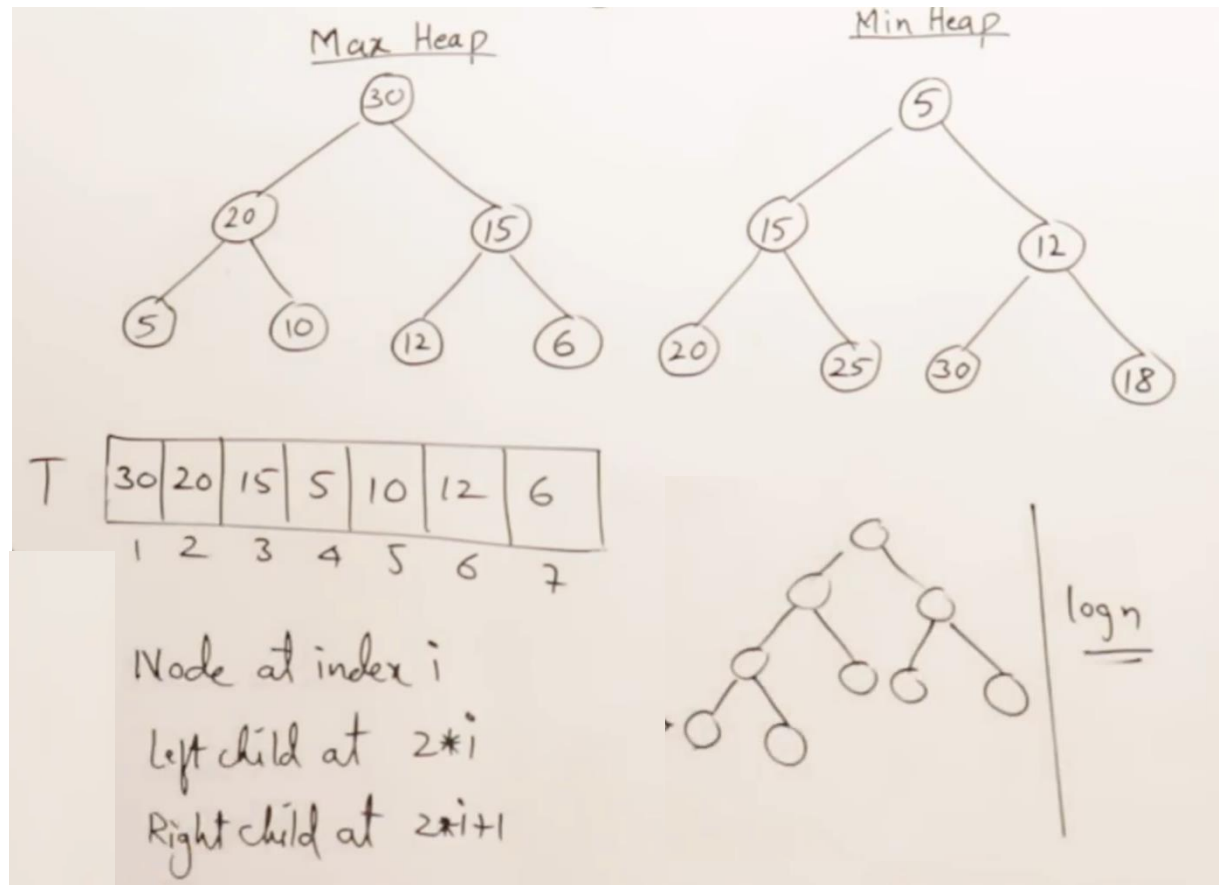


Height of the BST depends on the order of the insertion of nodes.

Heaps and priority queues

- Sometimes we don't need a completely sorted structure. Rather, we just want to get the highest priority item each time.
- A heap is *complete binary tree* with one of the following two properties...
 - Max-heap: every node stores a value greater than or equal to those of its children (no order imposed on children)
 - Min-heap: every node stores a value less than or equal to those of its children

Heaps



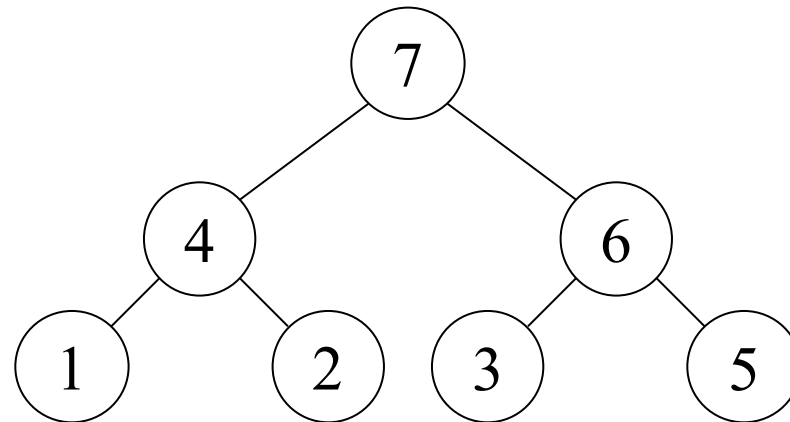
Heaps are usually represented using arrays.

Height is always $\log n$ since heaps are complete binary trees.

Heaps are not useful for searching purpose.

Array representation

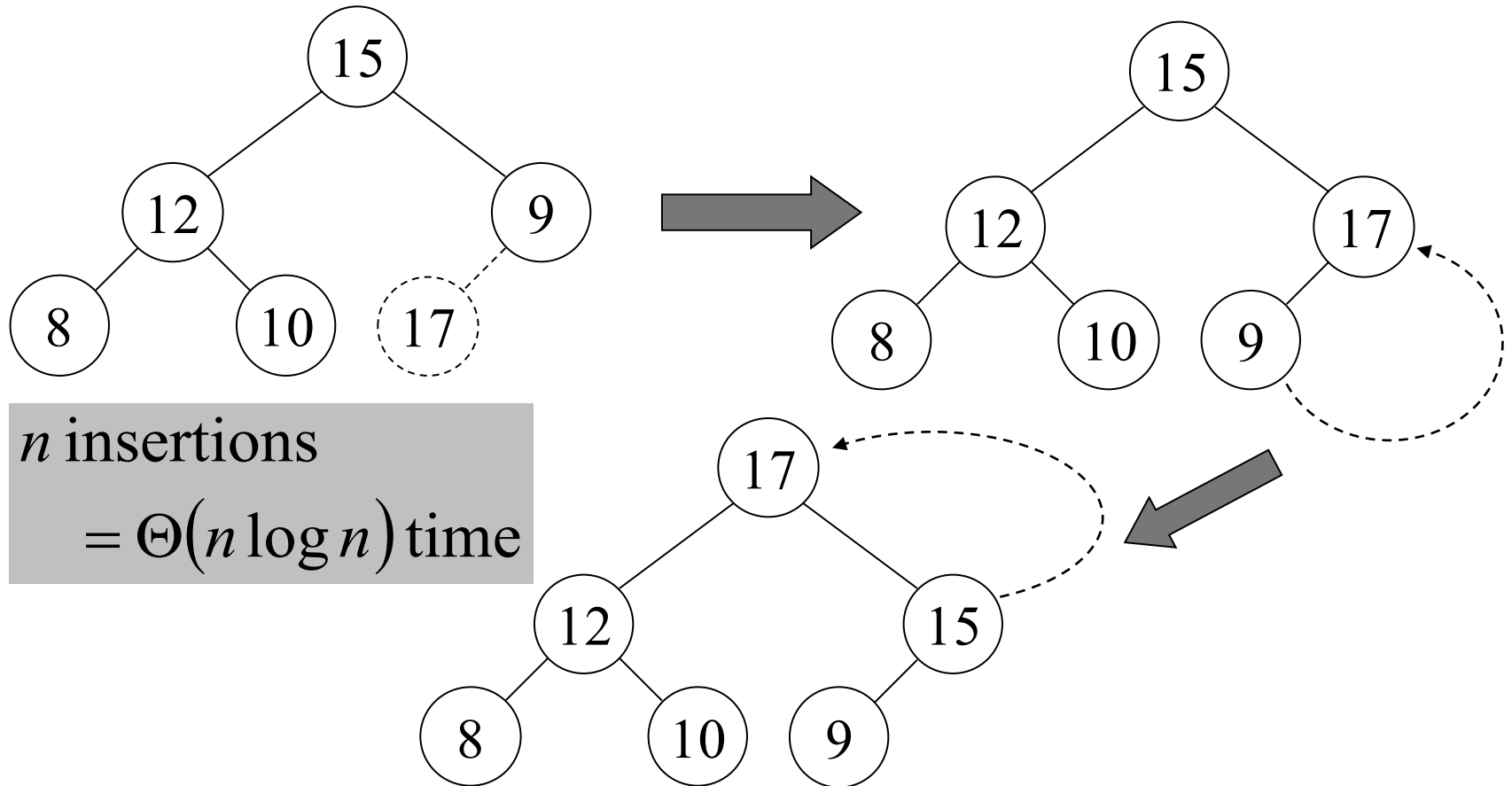
- Since a heap is always complete, we can use the array representation for space savings



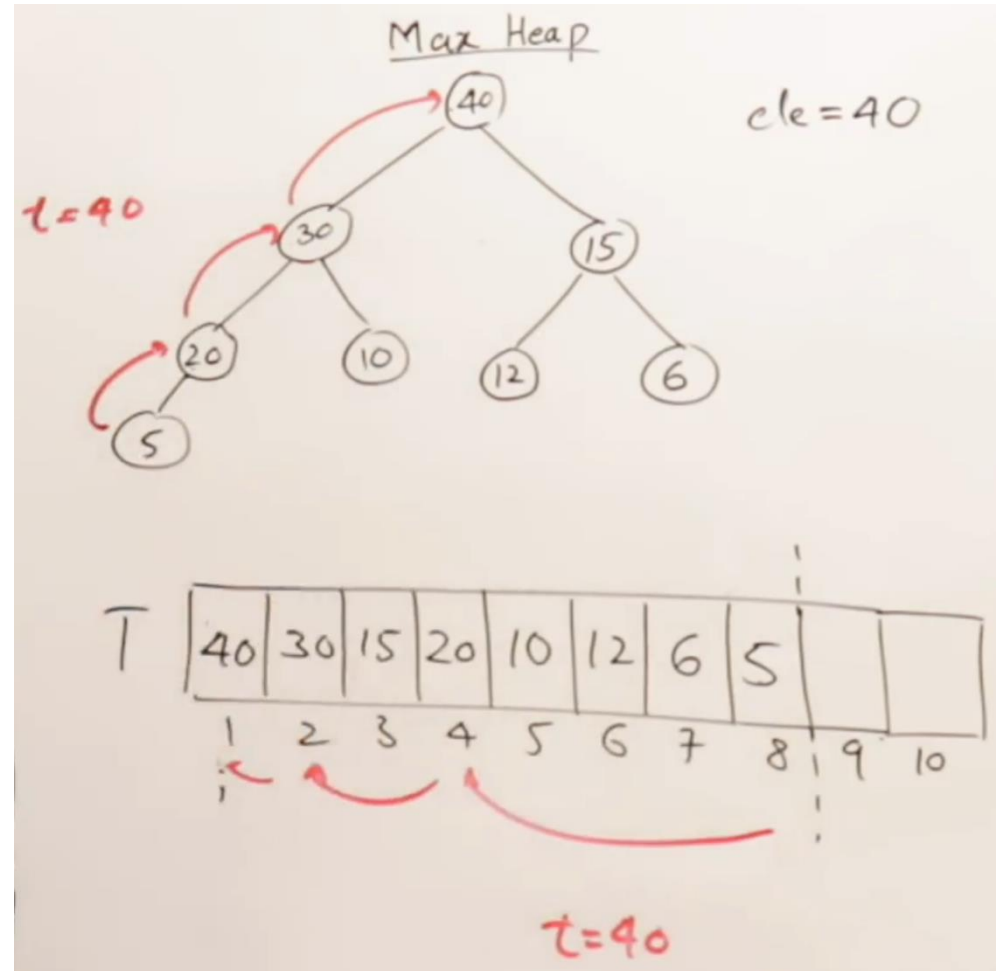
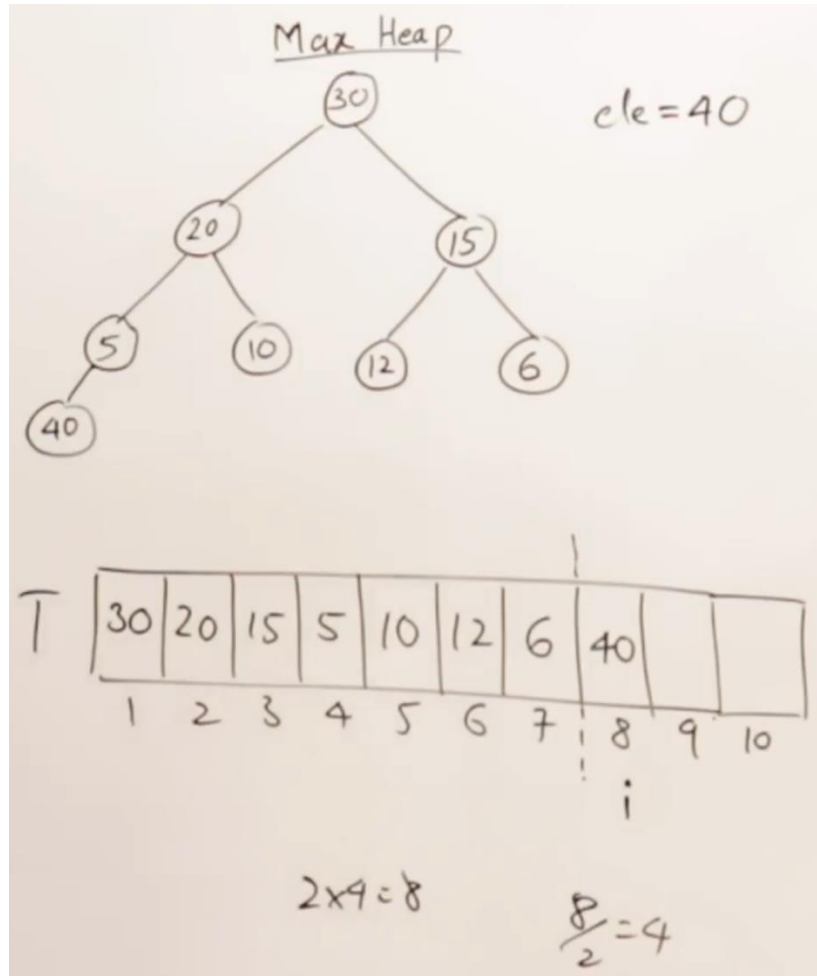
7	4	6	1	2	3	5
---	---	---	---	---	---	---

Heap insert

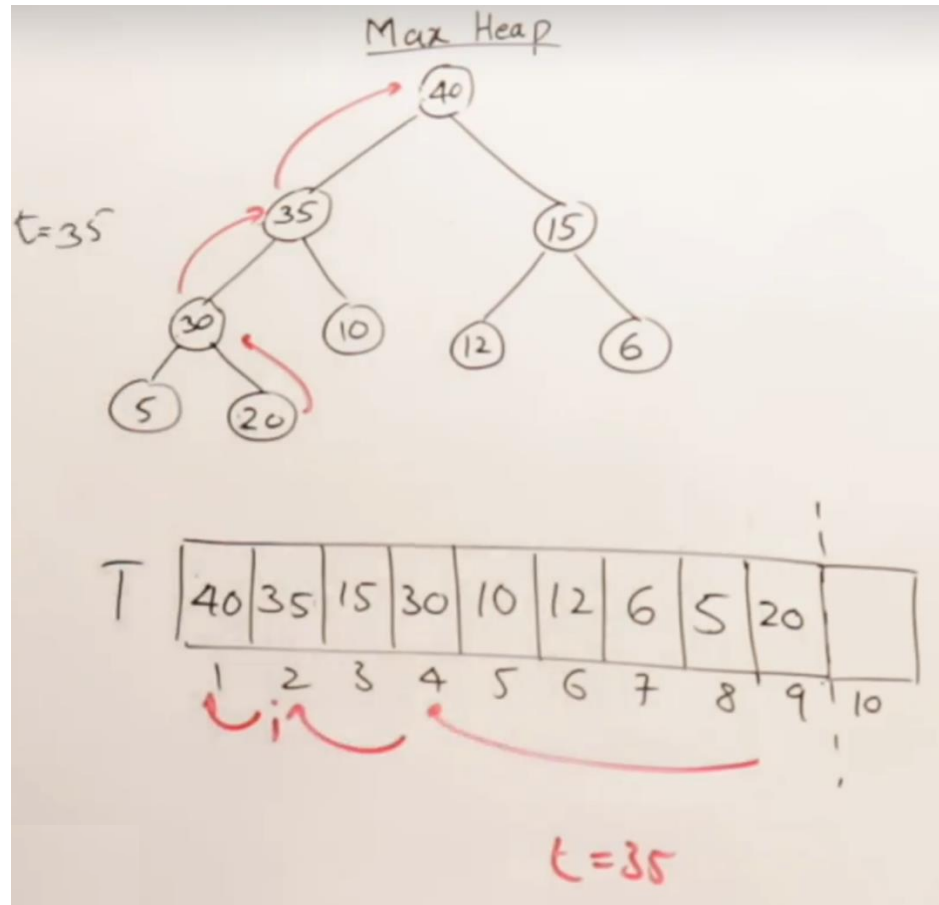
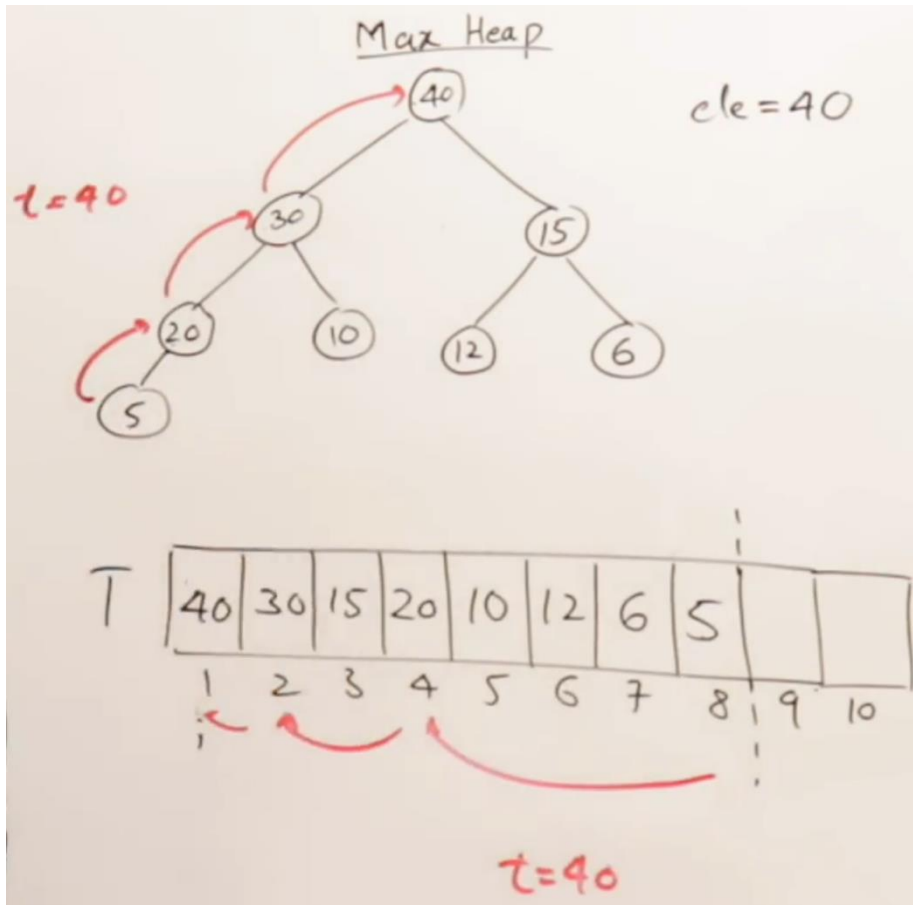
- Add an element at the end of the heap
- While (greater than parent) {swap};



Heap insert



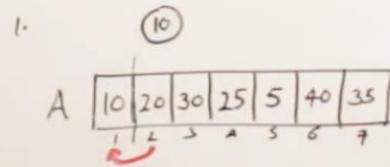
Heap insert



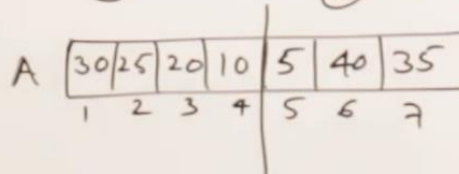
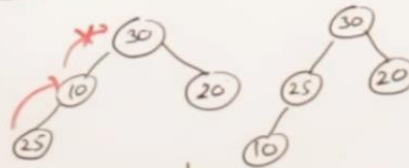
Heap insert

Binary Heap

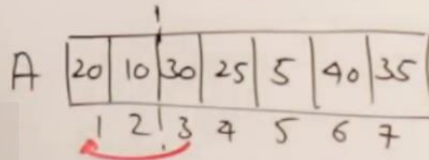
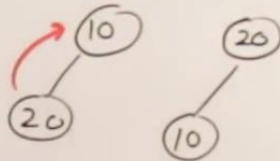
```
void Insert(int A[], int n)
{
    int temp, i = n;
    temp = A[n];
    while (i > 1 && temp > A[i/2])
    {
        A[i] = A[i/2];
        i = i/2;
    }
    A[i] = temp;
}
```



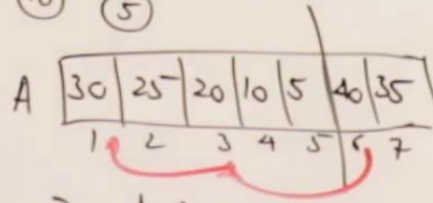
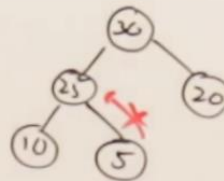
4. Insert 25



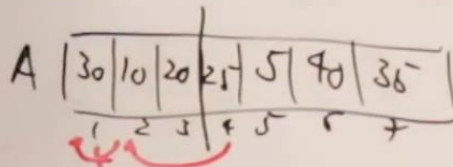
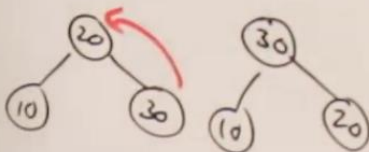
2. Insert 20



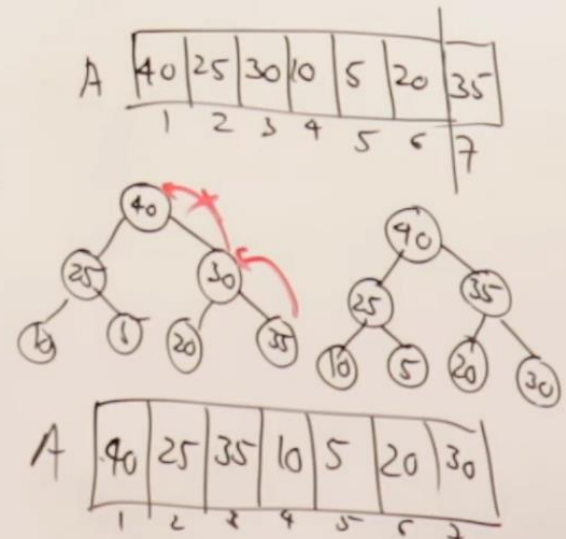
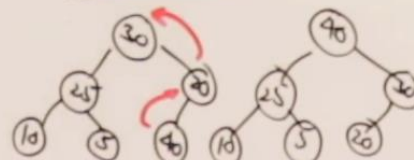
5. Insert 5



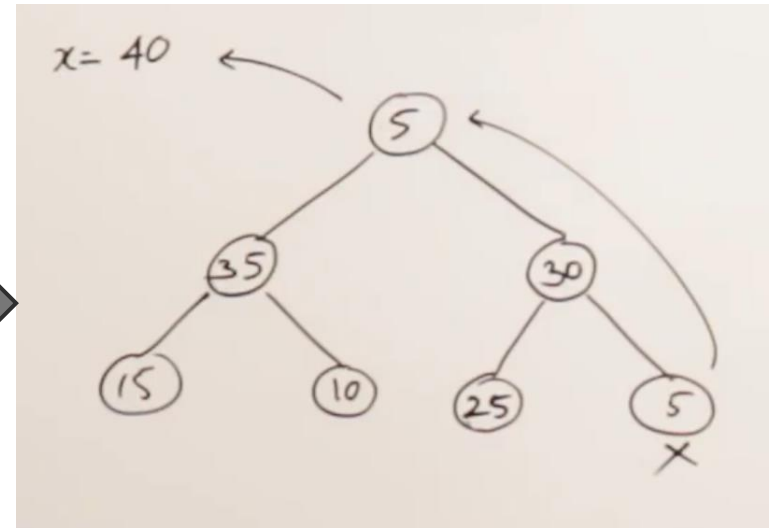
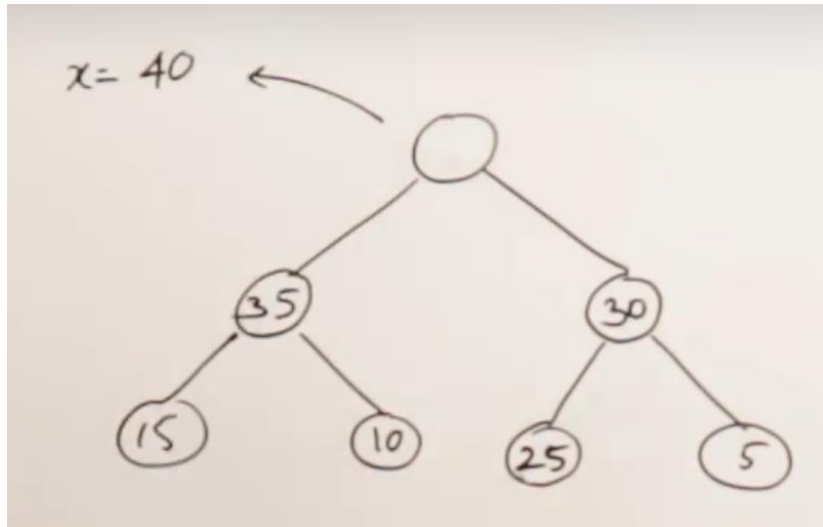
3. Insert 30



6. Insert 40

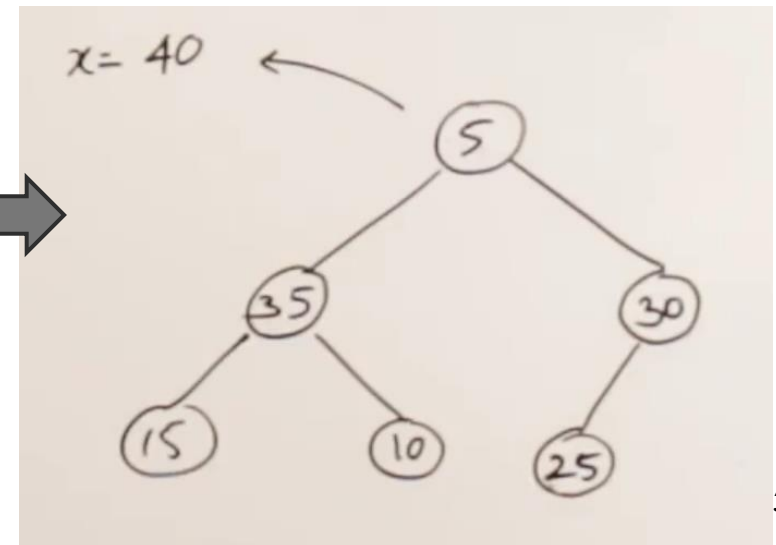


Heap delete

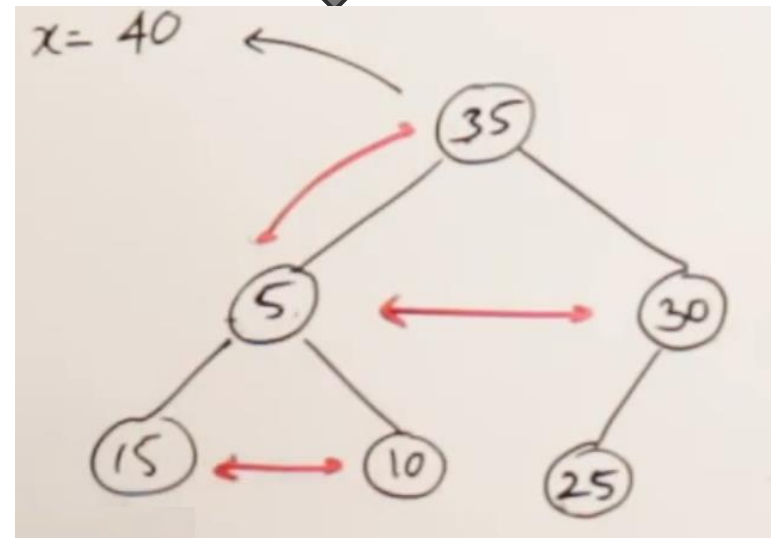
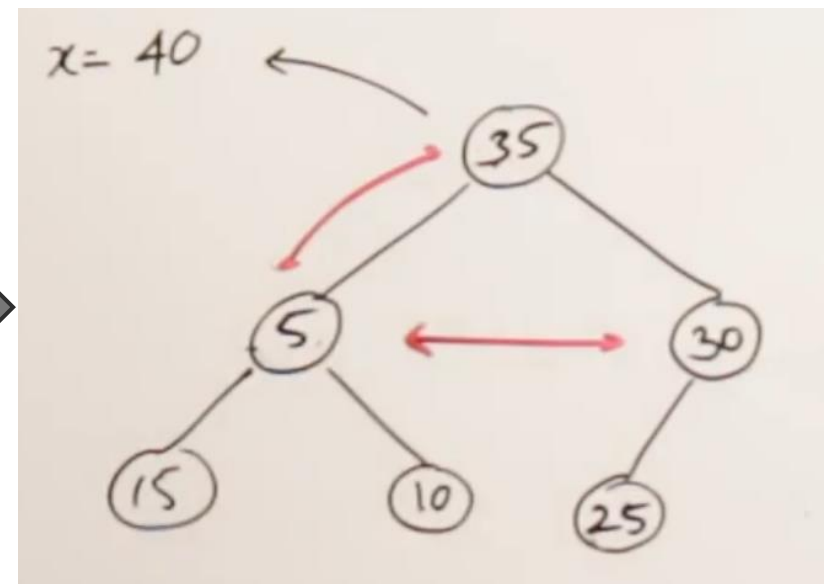
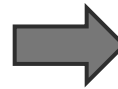
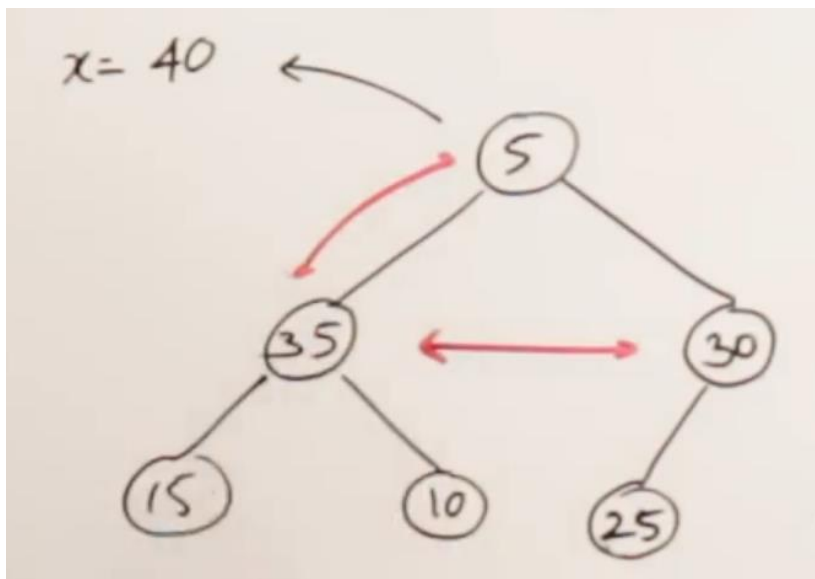


Only root is deleted

- ❑ A complete binary tree but not max heap.
- ❑ We need to move 5 to its correct location.

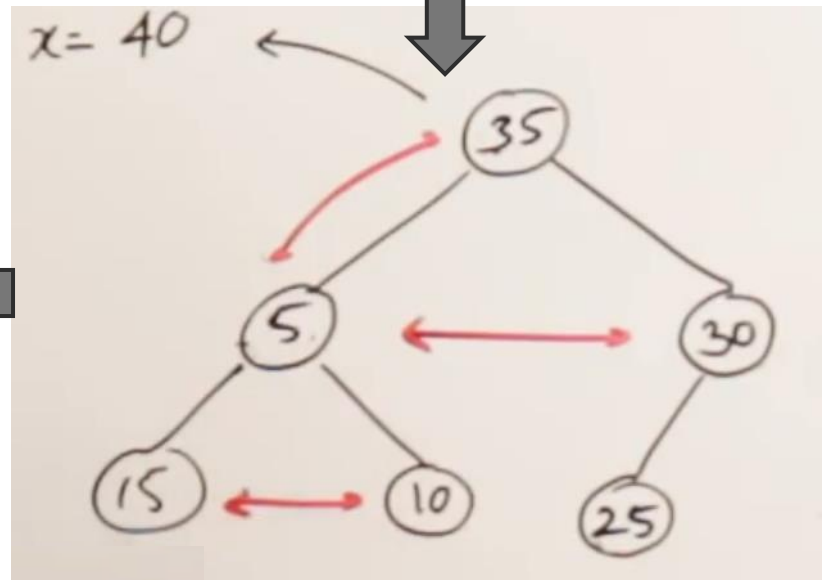
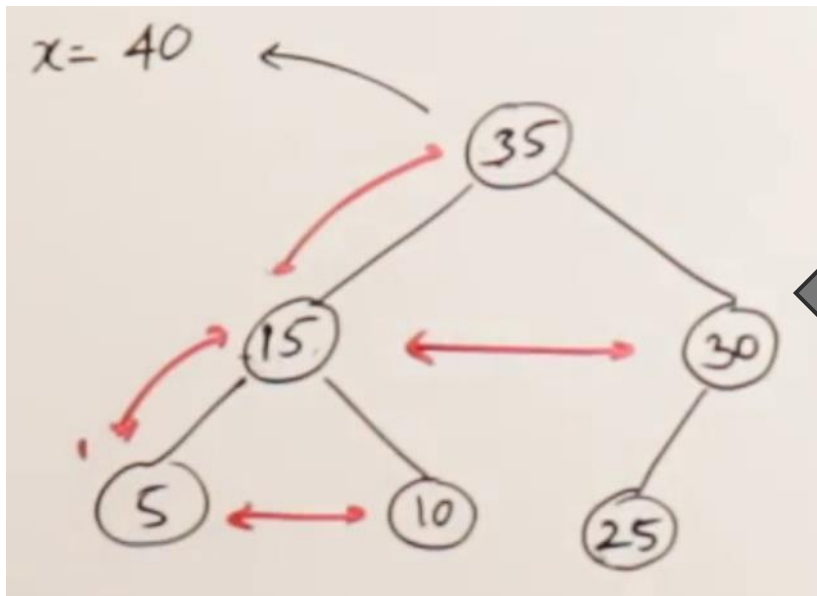
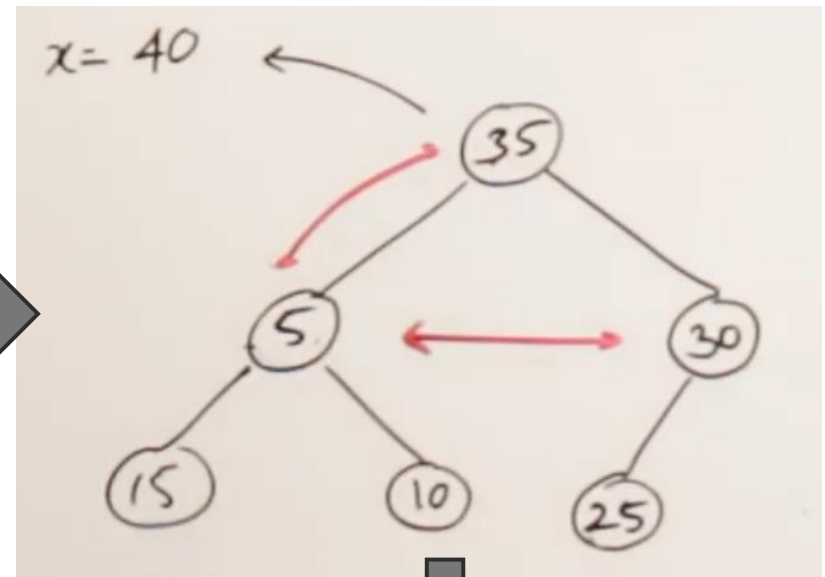
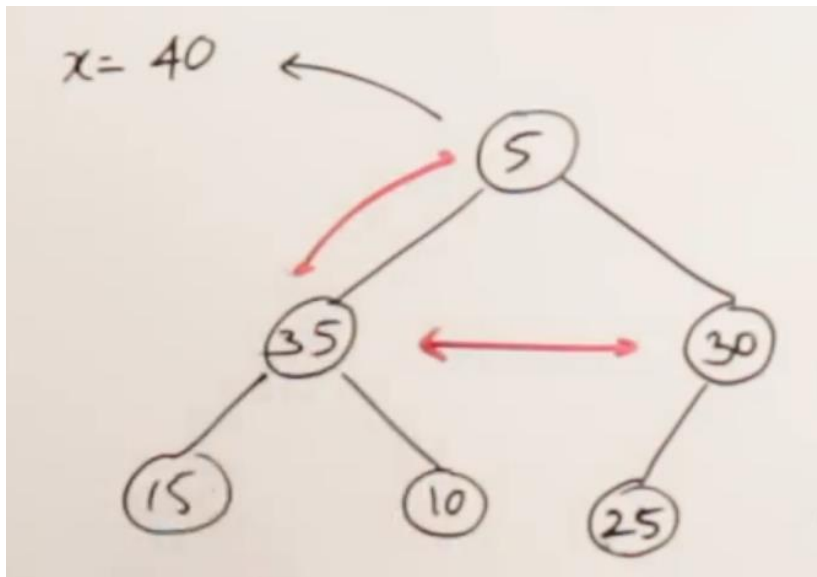


Heap delete



- ☐ Compare children of the new root.
- ☐ Compare the greater child with the new root.
- ☐ If the root is smaller, then swap.

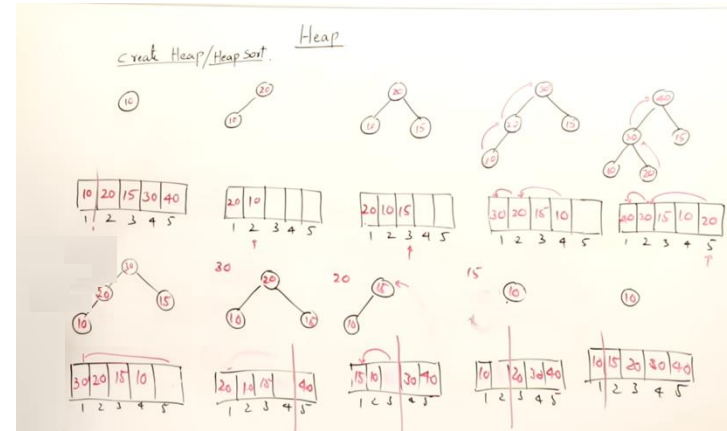
Heap delete



Heap Sort

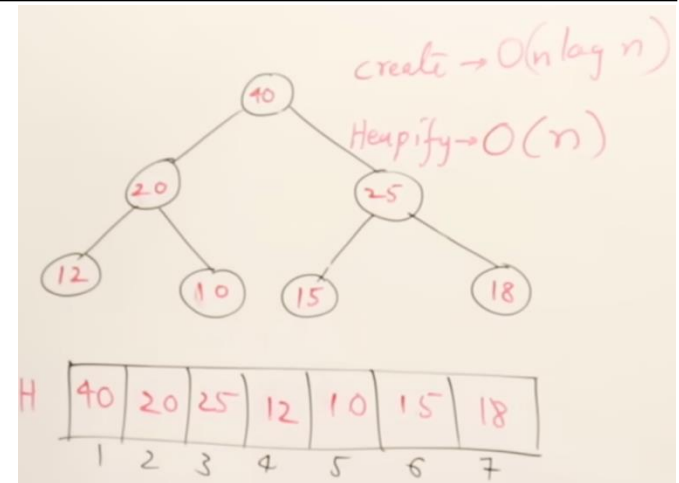
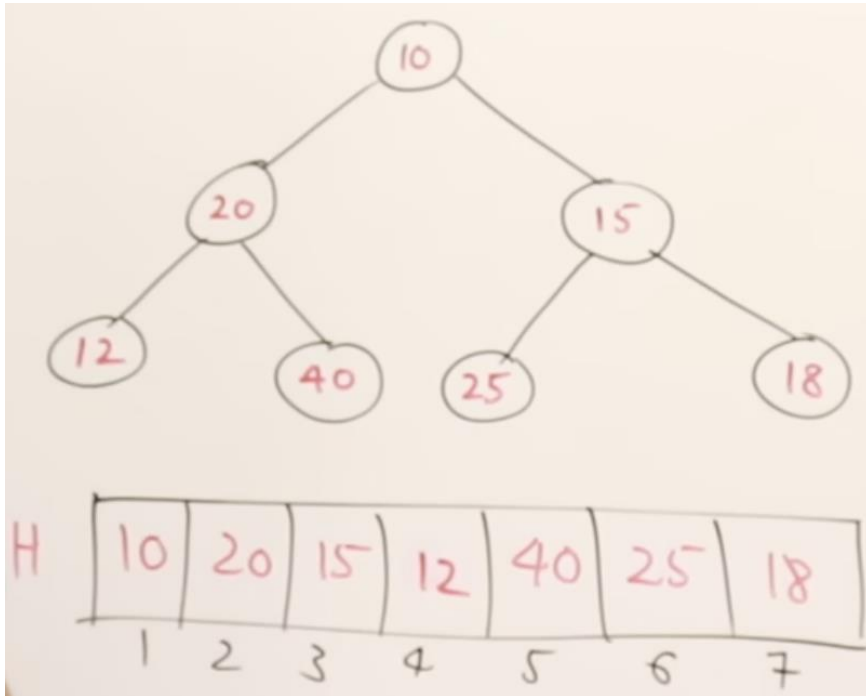
Steps:

- ❑ Create a heap given a set of keys.
- ❑ Delete the top node (root) one-by-one, and add the deleted node to the emptied spot in the array.



Heapify

- ❑ Process of creating a heap given a set of keys using bottom-up processing.



The whole point

- The most important operation: remove and return the max-priority element
 - Can't remove the root and maintain a complete binary tree shape
 - The only element we can remove is the last element
 - Swap last with root and `sift down()`
 - $\Theta(\log n)$ in average and worst cases
- Changing priorities: not efficient to find arbitrary elements, only the top.
 - Use an additional data structure (BST) with pointers

Coding schemes

- ASCII – a fixed length coding scheme:

ASCII value	Character	Control character	ASCII value	Character	ASCII value	Character	ASCII value	Character
000	(null)	NUL	032	(space)	064	@	096	
001	☺	SOH	033	!	065	A	097	a
002	☹	STX	034	"	066	B	098	b
003	♥	ETX	035	#	067	C	099	c
004	♦	EOT	036	\$	068	D	100	d
005	♣	ENQ	037	%	069	E	101	e
006	♠	ACK	038	&	070	F	102	f
007	(beep)	BEL	039	'	071	G	103	g
008	■	BS	040	(	072	H	104	h
009	(tab)	HT	041	)	073	I	105	i
010	(line feed)	LF	042	*	074	J	106	j
011	(home)	VT	043	+	075	K	107	k
012	(form feed)	FF	044	,	076	L	108	l
013	(carriage return)	CR	045	-	077	M	109	m
014	♪	SO	046	.	078	N	110	n
015	☼	SI	047	/	079	O	111	o
016	▲	DLE	048	0	080	P	112	p
017	▼	DC1	049	1	081	Q	113	q
018	↕	DC2	050	2	082	R	114	r
019	!!!	DC3	051	3	083	S	115	s
020	π	DC4	052	4	084	T	116	t
021	\$	NAK	053	5	085	U	117	u
022	▬	SYN	054	6	086	V	118	v
023	↕	ETB	055	7	087	W	119	w
024	↕	CAN	056	8	088	X	120	x
025	↕	EM	057	9	089	Y	121	y
026	→	SUB	058	:	090	Z	122	z
027	←	ESC	059	:	091	[	123	{
028	(cursor right)	FS	060	<	092	\	124	
029	(cursor left)	GS	061	=	093	]	125	}
030	(cursor up)	RS	062	>	094	^	126	~
031	(cursor down)	US	063	?	095	_	127	☐

Copyright 1998, JimPrice.Com Copyright 1982, Loading Edge Computer Products, Inc.

Variable length scheme

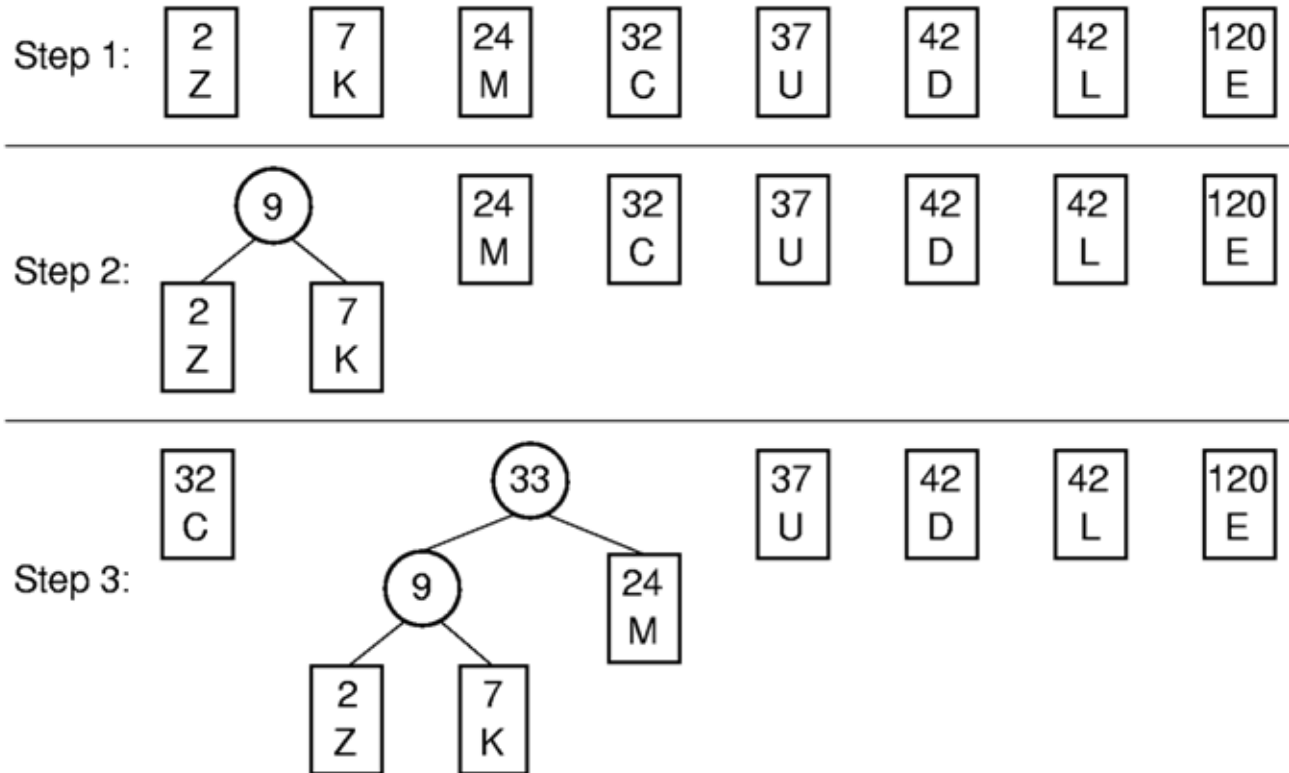
- In general, letters such as *s*, *i*, and *e* are used more often than, say *z*.
- If the code for *s* is 01, and the code for *z* is 1001011, we might be able to save some space
 - No other code can start with 01, or we will think it is an *s*

The Huffman Tree

- A full binary tree with letters at the leaf nodes and labeled edges
- Assign weights to the letters that represent how often they are used
 - s – high weight, z – low weight
- *How do we decide how often? What type of communications?*

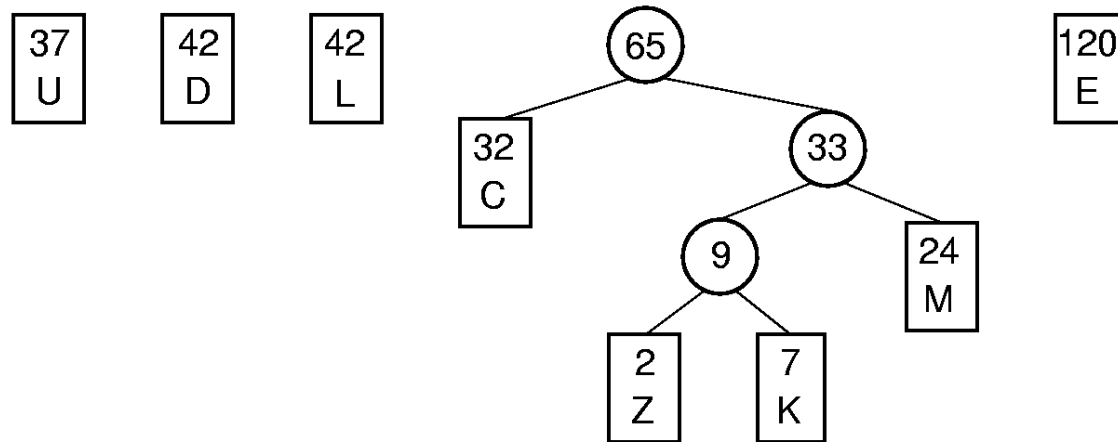
Building a Huffman tree

- To start, each character is its own (full) binary tree
- Merge trees with lowest weights
- New weight is sum of previous weights
- Continue...

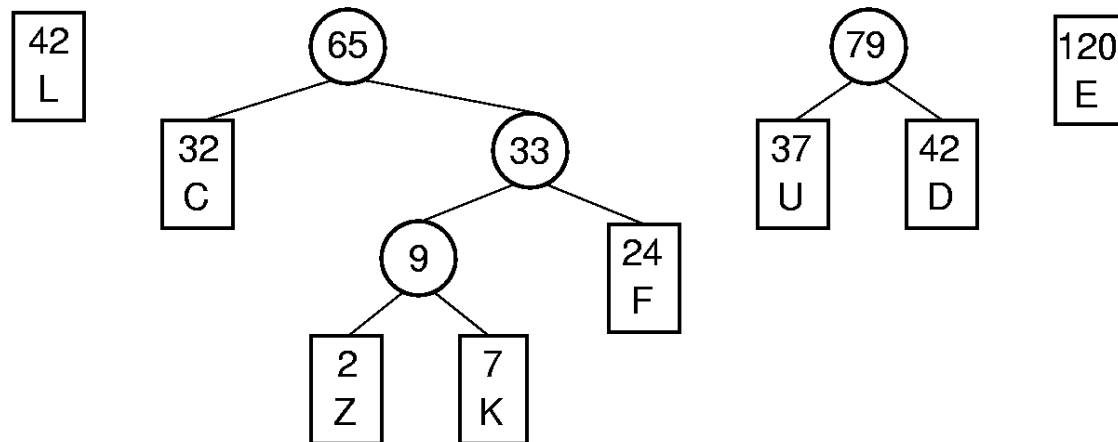


Building a Huffman tree (2)

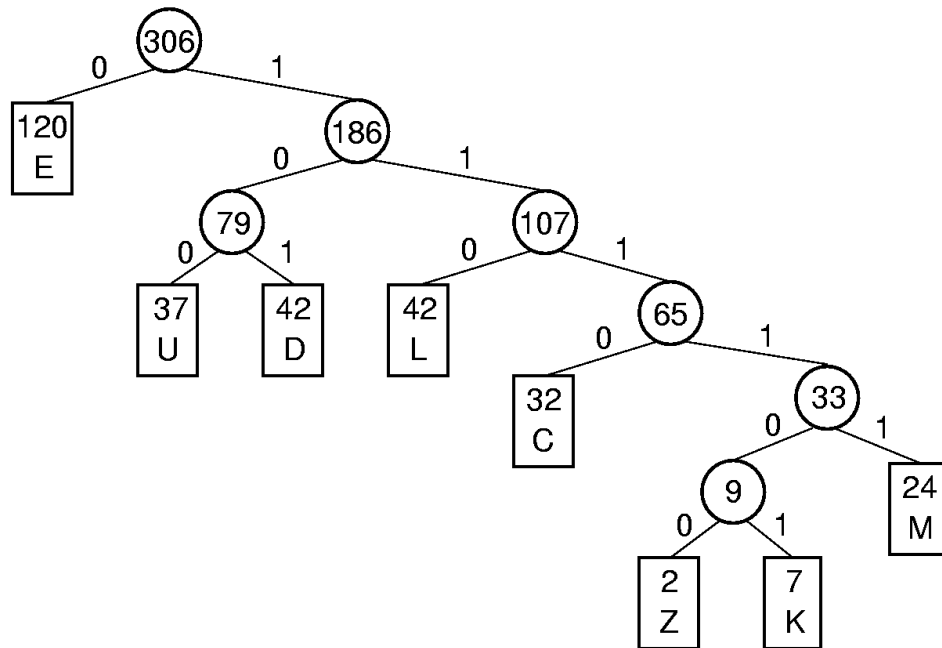
Step 4:



Step 5:



Assigning codes



Letter	Freq	Code	Bits
C	32		
D	42		
E	120		
F	24		
K	7		
L	42		
U	37		
Z	2		

Properties of Huffman codes

- No letters at internal nodes, so no character has a code that is a prefix of another character's code.
- What if we write a message with a lot of z's?
- Average character cost:

$$c_1p_1 + c_2p_2 + \cdots + c_np_n \quad \text{where} \quad p_i = f_i / f_T$$

$$= \frac{c_1f_1 + c_2f_2 + \cdots + c_nf_n}{f_T}$$